Research and Teaching Extensions to the PSAT 2.1.11 Power System Analysis Toolbox

A Technical Report on Architecture, Mathematical Formulations, Verification Methodology, and Test Results

Prof. Robert A. Sowah
Control and Optimization in Energy Networks
Power Systems Laboratory, PSAT-Based Toolbox Extensions Project

19 October 2026

Abstract

PSAT 2.1.11 (Power System Analysis Toolbox, F. Milano) is a mature MATLAB-based research tool for power-flow, continuation power-flow, optimal power-flow, and time-domain/small-signal dynamic analysis. As shipped, it has no documented data schema, no structured results API, no distributed or uncertainty-aware optimization capability, no storage or grid-forming-inverter device model, no cyberattack or market-economics tooling, and no machine-learning or reinforcement-learning interface. This report documents a coordinated set of extensions – 25 Python modules, 31 MATLAB functions, and two genuine new PSAT device classes registered into PSAT's own class system (`@ESSclass`, `@GFMclass`) – built to close these gaps for graduate teaching and for modern power-systems research, without modifying any of PSAT's existing, working behaviour. Every capability reported here was verified against an independent method, a known closed-form result, or a live PSAT/MATLAB session before being accepted; this report documents not only the final, working mathematics but, where instructive, the genuine defects found and corrected during development, since several of them recur across more than one module and are worth recording as general lessons. Beyond stating each extension's governing equations, this report develops their theoretical basis formally: a dedicated Mathematical Preliminaries chapter establishes the convex-duality, fixed-point, graph-spectral, probabilistic, and optimal-control results every later chapter draws on, and each subsequent chapter states its own load-bearing claims as numbered Theorems, Lemmas, Propositions, and Corollaries with proofs – 36 such results in total, including a formal Bertsimas–Sim robust-counterpart derivation, an exact two-directional characterization of undetectable false-data-injection attacks, a Sherman–Morrison derivation of the line-outage distribution factor, a Riccati-recursion proof that receding-horizon MPC converges to the infinite-horizon LQR gain, and a Routh–Hurwitz derivation of the multi-machine small-signal 90° instability threshold – referencing standard graduate texts (Boyd & Vandenberghe, Bertsekas, Horn & Johnson, Abur & Expósito, Kundur, and others) throughout, rather than asserting results without derivation. The report is organised by capability area: mathematical preliminaries, core workflow/schema extensions, deep-integration device classes, distribution and fault analysis, optimization and dispatch under uncertainty, security and resilience, control, learning-based methods, uncertainty quantification, and MATPOWER interoperability. A full nomenclature table, rigorously derived mathematical formulations, and consolidated test-case results accompany each area.

Contents

1 Introduction **3**
- 1.1 Motivation . . . 3
- 1.2 Design principles . . . 3
- 1.3 Report structure . . . 4

2 Nomenclature **5**
- 2.1 Network and power-flow quantities . . . 5
- 2.2 Optimization quantities . . . 5
- 2.3 Dynamics and control quantities . . . 6
- 2.4 State estimation and security quantities . . . 6
- 2.5 Gas-network and thermal-rating quantities . . . 6
- 2.6 Learning-based-method quantities . . . 7

3 Mathematical Preliminaries **8**
- 3.1 Convex optimization, Lagrangian duality, and KKT conditions . . . 8
- 3.2 Fixed-point and contraction-mapping results . . . 9
- 3.3 Structured linear algebra on graphs . . . 9
- 3.4 Probability theory for Gaussian and quadratic-form random variables . . . 10
- 3.5 Discrete-time optimal control . . . 11

4 System Architecture and Data Schema **12**
- 4.1 The schema layer . . . 12
- 4.2 The Python I/O layer: `psat_io.py` . . . 13
- 4.3 The structured-results layer: `psat_results.m` . . . 14
- 4.4 Parameter sweeps and physically-modified data files . . . 14
- 4.5 N-1 classification and secure ATC . . . 15
- 4.6 Linearization export for control design . . . 16
- 4.7 Automated lab-report generation: `lab_report_generator.py` . . . 17

5 Deep-Integration Device Classes **18**
- 5.1 `@ESSclass`: a battery/storage device . . . 19
- 5.2 `@GFMclass`: a grid-forming inverter (virtual synchronous machine) . . . 20

6 Distribution, Fault, and Multi-Energy Network Analysis **23**
- 6.1 Three-phase unbalanced distribution power flow . . . 24
- 6.2 Short-circuit fault analysis . . . 26
- 6.3 Gas-pipeline networks: the Weymouth equation . . . 28
- 6.4 Network visualization . . . 30

7 Optimization and Economic Dispatch Extensions **31**
7.1 Distributed optimization: dual decomposition, ADMM, and consensus 32
7.2 Stochastic and robust dispatch under uncertainty 35
7.3 Battery/storage arbitrage dispatch 38
7.4 Market clearing, congestion pricing, and real options 40
7.5 A MOST-equivalent multi-period scheduling tool 42
7.6 Stability-constrained optimal power flow 44

8 Security and Resilience Extensions **47**
8.1 DC state estimation and stealthy false-data injection 48
8.2 Extended attack surface: GPS spoofing and a measurement-protection game 50
8.3 Cascading failure and N-k analysis 52
8.4 Decentralized wide-area frequency control 53

9 Control Extensions **54**
9.1 MPC and LQR from a PSAT linearization 54

10 Learning-Based Methods **57**
10.1 A graph neural network surrogate for AC power flow 58
10.2 A reinforcement-learning battery-scheduling environment 60

11 Uncertainty Quantification **62**
11.1 Probabilistic power flow 63
11.2 Dynamic line rating (IEEE 738) 65

12 Interoperability with MATPOWER **67**

13 Verification Methodology and Consolidated Defect Log **69**
13.1 Cross-language parity 69
13.2 Consolidated defect log 70

14 Known Limitations **72**

15 Conclusion and Future Work **73**

Chapter 1

Introduction

1.1 Motivation

PSAT 2.1.11 is widely used for teaching power-flow, continuation power-flow (voltage-stability), optimal power-flow, and time-domain/small-signal dynamic analysis, and it ships with roughly forty ready-made test systems. It is, however, a *research* tool first: its device classes (`@SYclass`, `@EXclass`, `@TGclass`, `@PQclass`, ...) have undocumented column schemas; its results are scattered across global variables with no structured API; its optimization capability (`opf`) is a single deterministic, centralized solve; and it has no native model at all for battery storage, grid-forming inverters, gas-electric coupling, market economics, cyberattack surfaces, or learning-based surrogates – capabilities now standard in the modern power-systems research literature and in this course's own 16-week syllabus.

This project closes each of those gaps with a self-contained extension, built and verified in either Python (as a reusable library depending only on `numpy`/`scipy`/`cvxpy`/`networkx`/`torch`/ `gymnasium`) or MATLAB (as a function operating on PSAT's own loaded data, or as a PSAT-native device class), or both.

1.2 Design principles

Four principles were applied uniformly across every extension in this report:

1. **Additive, non-destructive integration.** No existing PSAT behaviour was changed. The two new device classes (§5) were wired into PSAT's core dispatch files by *adding* lines that reference the new device type; every original file was backed up before editing.

2. **Test before embed.** Every numeric claim in this report was produced by code that was actually executed, not assumed correct by inspection. Wherever an independent cross-check existed (a brute-force enumeration, a different solver, a closed-form textbook result, a second programming language), it was used.

3. **Honest reporting of negative and null results.** Several extensions report a finding that did *not* go the way a first intuition expected (§10, §8); these are reported as found, not adjusted to look better.

4. **Cross-language parity.** Every Python module with a natural PSAT-side analogue has a MATLAB port, tested live against a running MATLAB R2026a/PSAT 2.1.11 session (not merely written to look plausible). Table 13.1 in §13 gives the complete parity map.

1.3 Report structure

Chapter 2 is a complete nomenclature table. Chapter 3 collects, with full proofs or precise citations, every piece of convex-optimization, fixed-point, graph-linear- algebra, probability, and optimal-control theory invoked more than once later in the report, so that each later chapter can cite a numbered result rather than re-derive it. Chapter 4 describes the data schema and I/O layer every other extension builds on. Chapters 5–12 cover each capability area in turn: mathematical formulation (stated, where the result is load-bearing, as a numbered Theorem/Lemma/Proposition with a proof, not only an equation), algorithm, and verified test-case results. Every Theorem, Lemma, Proposition, and Corollary in the report shares one numbering sequence per chapter (e.g. Theorem 7.6 is the one and only "Theorem 7.6"), so a forward or backward cross-reference always resolves unambiguously. Chapter 13 gives the cross-language parity map and a consolidated log of every genuine defect found during development – a resource in its own right, since several of these defects (sign-convention bugs, slack-bus formulation errors, nested-function closures) recur in predictable, nameable ways. Chapter 14 states every known limitation plainly. Chapter 15 concludes.

Chapter 2

Nomenclature

Symbols are grouped by the subject area in which they are used; several letters (e.g. H, K) are reused across areas with different meanings, as is standard in the power-systems literature – the local meaning is always restated at first use within each section.

2.1 Network and power-flow quantities

Symbol	Meaning
n, m	Number of buses; number of branches/lines.
θ_i	Voltage phase angle at bus i (rad).
V_i	Voltage magnitude at bus i (pu).
B, B'	Full and reduced (non-slack) nodal susceptance (Laplacian-like) matrix.
Y_{bus}, Z_{bus}	Bus admittance matrix and its inverse, the bus impedance matrix.
$P_{g,i}$, $P_{d,i}$	Real power generation and demand at bus i.
PTDF	Power Transfer Distribution Factor matrix, $\partial(\text{flow})/\partial(\text{injection})$.
$\text{LODF}_{l,k}$	Line Outage Distribution Factor: fraction of outaged line k's pre-contingency flow redistributed onto monitored line l.
$\text{rate}A$	Thermal flow rating of a line (MW or pu).

2.2 Optimization quantities

Symbol	Meaning
a_i, b_i	Quadratic and linear generator cost coefficients, $\text{cost}_i(P) = a_i P^2 + b_i P$.
λ	Lagrange multiplier / shadow price of a power-balance equality constraint (the system marginal cost, or locational marginal price, LMP).
Γ	Bertsimas–Sim "budget of uncertainty" (number of uncertain parameters simultaneously allowed to reach their worst case).
$\hat{d}_i$	Known deviation magnitude of uncertain parameter i; true value $d_i = d_i^{\text{nom}} + \xi_i \hat{d}_i$, $\xi_i \in [-1, 1]$.

Symbol	Meaning
ρ	ADMM penalty parameter.
z, u	ADMM global consensus variable and scaled dual variable.
W	Metropolis–Hastings consensus weight matrix (symmetric, doubly stochastic).
VSS	Value of the Stochastic Solution (expected-value solution cost minus two-stage recourse cost).

2.3 Dynamics and control quantities

Symbol	Meaning
H_i	Inertia constant of machine/virtual machine i (s).
D_i	Damping coefficient of machine/area i (pu/pu).
ω	Per-unit rotor (or virtual-rotor) speed, nominal 1.
$K_{ij}(\theta^\star)$	Synchronizing-power (small-signal stiffness) coefficient, $B_{ij}\cos(\theta_i^\star - \theta_j^\star)$.
A, B, C, D (state space)	Continuous-time linear state-space matrices, $\dot{x} = Ax + Bu$, $y = Cx + Du$.
A_d, B_d	Zero-order-hold discretized equivalents of A, B.
Q, R_w	State and input quadratic-cost weighting matrices for LQR/MPC.
K_{lqr}	Infinite-horizon LQR gain, from the discrete algebraic Riccati equation (DARE).
ACE_k	Area control error of area k: tie-line flow deviation plus frequency-bias term.

2.4 State estimation and security quantities

Symbol	Meaning
H (measurement)	Measurement Jacobian, $m \times n$ (m measurements, n state variables).
z (measurement)	Measurement vector.
R	Measurement noise covariance (diagonal).
$\hat{x}$	Weighted-least-squares state estimate.
J	Chi-square bad-data detection statistic, $J = r^\top R^{-1} r$.
a	Measurement attack vector; a *stealthy* attack satisfies $a = Hc$ for some state-shift vector c.

2.5 Gas-network and thermal-rating quantities

Symbol	Meaning
p_i	Nodal gas pressure (pressure plays the role voltage plays in power flow).
K_{ij} (Weymouth)	Weymouth constant of pipe (i, j).
q_{ij}	Steady-state gas flow on pipe (i, j).
$I_{\max}$	Dynamic conductor ampacity (A), from the IEEE 738 heat balance.
q_c, q_r, q_s	Convective, radiative, and solar heat flux terms (W/m).
$R(T_c)$	Conductor AC resistance at conductor temperature T_c.

2.6 Learning-based-method quantities

Symbol	Meaning
x_v	Node feature vector (GNN).
e_{uv}	Edge feature vector (GNN).
msg_{uv}	Message computed on directed edge $u \to v$.
agg_v	Sum-aggregated incoming messages at node v.
s_t, a_t, r_t	RL state, action, and reward at time t.
γ_{RL}	Reinforcement-learning discount factor.

Chapter 3

Mathematical Preliminaries

Every later chapter invokes one or more of five bodies of theory: convex optimization and duality, fixed-point/contraction arguments, structured linear algebra on graphs, probability theory for Gaussian and quadratic-form random variables, and discrete-time optimal control. This chapter states each result once, in the generality needed later, with a proof or a precise citation to a standard graduate reference, so that every subsequent chapter can invoke it by number rather than re-deriving it. Readers already fluent in this material may skip to Chapter 4; readers who want full proofs beyond the sketches given here are pointed to the cited source at each result.

3.1 Convex optimization, Lagrangian duality, and KKT conditions

Definition 3.1 (Convex set and convex function)**.** A set $C \subseteq \mathbb{R}^n$ is *convex* if $\lambda x + (1-\lambda)y \in C$ for all $x, y \in C$, $\lambda \in [0,1]$. A function $f : C \to \mathbb{R}$ is convex if its epigraph is convex, equivalently $f(\lambda x + (1-\lambda)y) \le \lambda f(x) + (1-\lambda) f(y)$ for all $x, y \in C$, $\lambda \in [0,1]$; f is *strictly convex* if the inequality is strict whenever $x \neq y$, $\lambda \in (0,1)$.

Definition 3.2 (Primal problem and Lagrangian)**.** Consider

$$\min_{x \in \mathbb{R}^n} \; f_0(x) \quad \text{s.t.} \quad g_i(x) \le 0 \; (i = 1, \dots, m), \quad h_j(x) = 0 \; (j = 1, \dots, p), \tag{3.1}$$

with f_0, g_i convex and h_j affine. The Lagrangian is $L(x, \mu, \nu) = f_0(x) + \sum_i \mu_i g_i(x) + \sum_j \nu_j h_j(x)$, $\mu \ge 0$, and the dual function is $d(\mu, \nu) = \inf_x L(x, \mu, \nu)$.

Assumption 3.3 (Slater's condition)**.** There exists a strictly feasible point $\tilde{x}$ with $g_i(\tilde{x}) < 0$ for every i and $h_j(\tilde{x}) = 0$ for every j.

Theorem 3.4 (Strong duality and KKT necessity)**.** *Under Assumption 3.3, if $x^\star$ solves the primal problem above then there exist $\mu^\star \ge 0, \nu^\star$ (dual optimal, attaining the primal optimal value with zero duality gap) such that the following Karush–Kuhn–Tucker (KKT) conditions hold:*

$$\nabla f_0(x^\star) + \textstyle\sum_i \mu_i^\star \nabla g_i(x^\star) + \sum_j \nu_j^\star \nabla h_j(x^\star) = 0 \quad \textit{(stationarity)} \tag{3.2}$$

$$g_i(x^\star) \le 0, \; h_j(x^\star) = 0 \quad \textit{(primal feasibility)} \tag{3.3}$$

$$\mu_i^\star \ge 0 \quad \textit{(dual feasibility)} \tag{3.4}$$

$$\mu_i^\star g_i(x^\star) = 0 \;\; \forall i \quad \textit{(complementary slackness)} \tag{3.5}$$

Conversely, for a convex problem, any (x, μ, ν) satisfying the KKT conditions is primal-dual optimal (sufficiency).

Proof sketch. Slater's condition rules out a duality gap for convex problems (Boyd & Vandenberghe [10], Ch. 5): strong duality $f_0(x^\star) = d(\mu^\star, \nu^\star)$ holds, and since $d(\mu^\star, \nu^\star) = \inf_x L(x, \mu^\star, \nu^\star) \le L(x^\star, \mu^\star, \nu^\star) = f_0(x^\star) + \sum_i \mu_i^\star g_i(x^\star) \le f_0(x^\star)$ (the last step using $\mu^\star \ge 0$, $g_i(x^\star) \le 0$), equality throughout forces both the complementary slackness condition above and that $x^\star$ minimizes $L(\cdot, \mu^\star, \nu^\star)$, which for differentiable f_0, g_i, h_j gives the stationarity condition above. Sufficiency for convex problems follows because the KKT stationarity condition makes x a global minimizer of the convex function $L(\cdot, \mu, \nu)$, and complementary slackness then equates $L(x, \mu, \nu)$ with $f_0(x)$. ■

Remark 3.5. Theorem 3.4 is the formal basis for every dual variable interpreted as a price or shadow price in this report (the LMP of §7.4, the reserve price of §7.2, the ADMM/dual-decomposition prices of §7.1): complementary slackness is exactly why a non-binding constraint has zero price and a binding constraint's price equals the constraint's marginal effect on the optimal objective (the envelope-theorem reading of the stationarity condition).

3.2 Fixed-point and contraction-mapping results

Definition 3.6 (Contraction)**.** A map $T : X \to X$ on a complete metric space (X, d) is a *contraction* with modulus $0 \le \gamma < 1$ if $d(T(x), T(y)) \le \gamma\, d(x, y)$ for all $x, y \in X$.

Theorem 3.7 (Banach fixed-point theorem)**.** *If $T : X \to X$ is a contraction on a complete metric space, then T has a unique fixed point $x^\star = T(x^\star)$, and for any $x_0 \in X$ the iterates $x_{k+1} = T(x_k)$ converge to $x^\star$ geometrically, $d(x_k, x^\star) \le \gamma^k d(x_0, x^\star)$.*

Proof. Standard; see Bertsekas [11], Appendix B, or any real analysis text. Existence/uniqueness follow from showing (x_k) is Cauchy (via the triangle inequality and $\sum \gamma^k < \infty$) and that the limit is fixed by continuity of T; the rate bound follows by induction on $d(x_{k+1}, x^\star) = d(T(x_k), T(x^\star)) \le \gamma\, d(x_k, x^\star)$. ■

Remark 3.8. Theorem 3.7 underlies two independent results used later: the Bellman-operator contraction behind tabular Q-learning/value iteration (§10.2), and, via the spectral-radius argument of §3.3, the geometric convergence of Metropolis– Hastings consensus averaging (§7.1).

3.3 Structured linear algebra on graphs

Definition 3.9 (Weighted graph Laplacian)**.** For a connected graph with n nodes, edge admittances $y_{ij} > 0$, the (weighted) Laplacian $B \in \mathbb{R}^{n \times n}$ has $B_{ii} = \sum_{j \sim i} y_{ij}$, $B_{ij} = -y_{ij}$ for $i \sim j$, 0 otherwise.

Theorem 3.10 (Laplacian rank and the reduced system)**.** *B is symmetric positive semi-definite with exactly one zero eigenvalue (eigenvector $\mathbf{1}$) if and only if the graph is connected. Consequently, deleting any one row/column (the reference bus) from B gives a principal submatrix B' that is symmetric positive definite, hence invertible.*

Proof. For any $x \in \mathbb{R}^n$, $x^\top B x = \sum_{i \sim j} y_{ij}(x_i - x_j)^2 \ge 0$, so $B \succeq 0$; equality holds iff $x_i = x_j$ for every edge, i.e. iff x is constant on each connected component, giving a null space of dimension equal to the number of components (one, under connectivity) – this is the standard Kirchhoff/Matrix-Tree argument (see Horn & Johnson [14], §13.4, or Godsil & Royle, *Algebraic Graph Theory*). A symmetric PSD matrix with a single zero eigenvalue has every principal submatrix obtained by deleting one row/column strictly positive definite, because the deleted-coordinate quadratic form $x'^\top B' x'$ (extending x' by a zero in the deleted coordinate) can only vanish for $x' = 0$, since the unique null direction $\mathbf{1}$ has a nonzero entry at the deleted coordinate. ■

Remark 3.11. Theorem 3.10 is why every DC-network formulation in this report (Chapters 7, 8) fixes one reference-bus angle and solves only the reduced system $B'\theta' = P'$: it is the formal reason B' is guaranteed nonsingular whenever the network is connected, matching the main course's own Theorem 1 treatment of the identical fact for AC power flow's own Y_{bus}-derived Jacobian structure.

Theorem 3.12 (Sherman–Morrison rank-one update)**.** *For invertible $A \in \mathbb{R}^{n\times n}$ and $u, v \in \mathbb{R}^n$ with $1 + v^\top A^{-1}u \neq 0$,*

$$(A + uv^\top)^{-1} = A^{-1} - \frac{A^{-1}uv^\top A^{-1}}{1 + v^\top A^{-1}u}. \tag{3.6}$$

Proof. Multiply the right-hand side by $(A + uv^\top)$ and simplify using $v^\top A^{-1}u$ as a scalar; the cross terms telescope to the identity. See Horn & Johnson [14], §0.7.4, or Golub & Van Loan [15], §2.1.3. ∎

Theorem 3.13 (Doubly stochastic consensus convergence)**.** *Let $W \in \mathbb{R}^{n\times n}$ be symmetric, doubly stochastic ($W\mathbf{1} = \mathbf{1}$, $W = W^\top$, $W \geq 0$ entrywise), and irreducible (the underlying communication graph is connected). Then the eigenvalues of W satisfy $1 = \mu_1 > \mu_2 \geq \cdots \geq \mu_n \geq -1$, and for any $x_0 \in \mathbb{R}^n$,*

$$W^k x_0 \;\longrightarrow\; \bar{x}\,\mathbf{1}, \qquad \bar{x} = \tfrac{1}{n}\textstyle\sum_i x_{0,i}, \tag{3.7}$$

at the geometric rate $\|W^k x_0 - \bar{x}\mathbf{1}\|_2 \leq |\mu_2|^k \, \|x_0 - \bar{x}\mathbf{1}\|_2$.

Proof. By the spectral theorem, symmetric W has an orthonormal eigenbasis; $\mathbf{1}$ is an eigenvector with eigenvalue 1 (row sums are 1). Since W is doubly stochastic its spectral radius is 1 (a stochastic matrix's eigenvalues lie in the closed unit disc by the Gershgorin circle theorem applied to its rows), and irreducibility plus the Perron–Frobenius theorem (see Horn & Johnson [14], Ch. 8, or Xiao & Boyd [16]) make 1 a simple eigenvalue with every other eigenvalue satisfying $|\lambda| < 1$ whenever W is also aperiodic (guaranteed here by positive diagonal entries, $W_{ii} > 0$, from the Metropolis–Hastings construction). Decomposing $x_0 = \bar{x}\mathbf{1} + e_0$ with $e_0 \perp \mathbf{1}$, $W^k x_0 = \bar{x}\mathbf{1} + W^k e_0$ and $\|W^k e_0\|_2 \leq |\mu_2|^k \, \|e_0\|_2 \to 0$. ∎

Remark 3.14. Theorem 3.13 is the exact reason this project's Metropolis–Hastings weight construction (§7.1) is required to be symmetric and doubly stochastic, not merely row- stochastic: a row-stochastic-only matrix generally has a LEFT Perron eigenvector $\pi \neq \mathbf{1}/n$, so $W^k x_0 \to (\pi^\top x_0)\mathbf{1}$, the π-weighted average, not the arithmetic mean – precisely the bug this project's own development documents (§7.1, Table 13.2) for both `decentralized_opt.py` and `@GFMclass`/`@ESSclass`'s consensus-dispatch counterpart.

3.4 Probability theory for Gaussian and quadratic-form random variables

Theorem 3.15 (Affine transformation of a Gaussian vector)**.** *If $X \sim \mathcal{N}(\mu, \Sigma)$ and $Y = AX + b$ for fixed $A \in \mathbb{R}^{m\times n}$, $b \in \mathbb{R}^m$, then $Y \sim \mathcal{N}(A\mu + b,\, A\Sigma A^\top)$.*

Proof. The characteristic function of X is $\varphi_X(t) = \exp(it^\top\mu - \tfrac{1}{2}t^\top\Sigma t)$; that of Y is $\varphi_Y(t) = \mathbb{E}[e^{it^\top(AX+b)}] = e^{it^\top b}\varphi_X(A^\top t) = \exp\big(it^\top(A\mu + b) - \tfrac{1}{2}t^\top(A\Sigma A^\top)t\big)$, which is exactly the characteristic function of $\mathcal{N}(A\mu + b, A\Sigma A^\top)$; a distribution is determined by its characteristic function (see Casella & Berger [17], Thm. 4.6.4, or any graduate probability text). ∎

Theorem 3.16 (Distribution of a quadratic form in Gaussian residuals)**.** *If $r \sim \mathcal{N}(0, \Sigma_r)$ with $\Sigma_r \succ 0$, then $J = r^\top \Sigma_r^{-1} r \sim \chi^2_k$ for some k depending on the rank structure of Σ_r; in particular, if $r = z - H\hat{x}$ is the weighted-least-squares residual of an m-measurement, n-state linear model with $\hat{x} = (H^\top R^{-1} H)^{-1} H^\top R^{-1} z$ and $\Sigma_r = R - H(H^\top R^{-1} H)^{-1} H^\top$, then $J \sim \chi^2_{m-n}$.*

Proof. Write $\Sigma_r = LL^\top$ (Cholesky); then $L^{-1} r \sim \mathcal{N}(0, I_k)$ (Theorem 3.15) and $J = \|L^{-1} r\|_2^2$ is a sum of k independent squared standard normals, the definition of χ^2_k. For the WLS residual, $r = (I - H(H^\top R^{-1} H)^{-1} H^\top R^{-1}) z$ is a linear projection of the Gaussian measurement noise onto an $(m-n)$-dimensional subspace orthogonal (in the R^{-1} inner product) to range(H), giving $k = m - n$ degrees of freedom; see Abur & Exposito [18], Ch. 5, for the full power-systems-state-estimation derivation. ∎

Remark 3.17. Theorem 3.16 is the formal basis for §8.1's chi-square bad-data test and, via Theorem 3.15 directly, for §11.1's exact analytical probabilistic power flow ($\Sigma_{\text{flow}} = \text{PTDF}\, \Sigma_{\text{inj}}\, \text{PTDF}^\top$ is nothing but Theorem 3.15 applied to the linear map flow $=$ PTDF $\cdot$ injections).

3.5 Discrete-time optimal control

Theorem 3.18 (Bellman's principle of optimality and the discrete Riccati recursion)**.** *For the linear-quadratic regulator problem $\min \sum_{k=0}^{N-1} (x_k^\top Q x_k + u_k^\top R_w u_k) + x_N^\top P_N x_N$ s.t. $x_{k+1} = A_d x_k + B_d u_k$, $Q \succeq 0$, $R_w \succ 0$, the optimal cost-to-go is quadratic, $V_k(x) = x^\top P_k x$, with P_k generated backward by the discrete Riccati recursion*

$$P_k = Q + A_d^\top P_{k+1} A_d - A_d^\top P_{k+1} B_d (R_w + B_d^\top P_{k+1} B_d)^{-1} B_d^\top P_{k+1} A_d, \qquad P_N \text{ given}, \tag{3.8}$$

and the optimal control is the linear state feedback $u_k^\star = -K_k x_k$, $K_k = (R_w + B_d^\top P_{k+1} B_d)^{-1} B_d^\top P_{k+1} A_d$. If (A_d, B_d) is stabilizable and $(A_d, Q^{1/2})$ detectable, $P_k \to P_\infty$ as the horizon recedes, where P_∞ is the unique stabilizing solution of the discrete algebraic Riccati equation (DARE), the fixed point of the Riccati recursion above.

Proof. By induction (Bellman's principle of optimality): assume $V_{k+1}(x) = x^\top P_{k+1} x$. Then $V_k(x) = \min_u \left[x^\top Q x + u^\top R_w u + (A_d x + B_d u)^\top P_{k+1} (A_d x + B_d u) \right]$ is an unconstrained convex quadratic in u (since $R_w + B_d^\top P_{k+1} B_d \succ 0$); setting its gradient to zero gives $u^\star = -K_k x$ with K_k as stated, and substituting back yields $V_k(x) = x^\top P_k x$ with P_k following the recursion above – closing the induction. Convergence of the backward recursion to the DARE's stabilizing fixed point under stabilizability/detectability is a classical result; see Anderson & Moore [19], Ch. 2, or Bertsekas [11], Vol. I, Ch. 4. ∎

Remark 3.19. Theorem 3.18 is restated here for self-containedness; it matches the main course's own Theorem 15 (proved there by the identical induction argument) and is the formal basis for §9.1's MPC-converges-to-LQR result.

With this toolkit in hand, every later chapter's Theorem/Proof blocks either specialize one of the results above or combine two or more of them; cross-references back to this chapter are given at each use.

Chapter 4

System Architecture and Data Schema

4.1 The schema layer

Every other extension in this report ultimately reads or writes PSAT's own `.con` device-data format. `schema/con_schema.json` was built by reading PSAT's own `@XXclass/XXclass.m` constructors directly (never guessed), giving an authoritative column count (`ncol`) for every device, and, for the highest-value devices this project's labs use in detail, a traced column-by-column meaning obtained by following the device's own `base.m`/`Fxcall.m` dynamics code. Each entry carries an explicit confidence level:

- **verified** – traced through source *and* cross-checked against a known real value (e.g. the WSCC-9 case's `Syn.con` column 18 gives $H = 23.64$ s, the published textbook inertia constant). Covers `Bus`, `Line`, `Syn`, `Exc`, `Tg` in full.
- **data_consistent** – `ncol`/position confirmed from source; meaning inferred from position and plausible data, not traced through the differential-equation code. Covers `SW`, `PV`, `PQ`, `Shunt`, `Fault`, `Breaker`, `Supply`, `Demand`.
- **unconfirmed** – `ncol` authoritative; column meaning not resolved. Covers roughly twenty lower-priority devices (FACTS, HVDC, wind/DFIG, PSS, load-modelling variants).

4.2 The Python I/O layer: `psat_io.py`

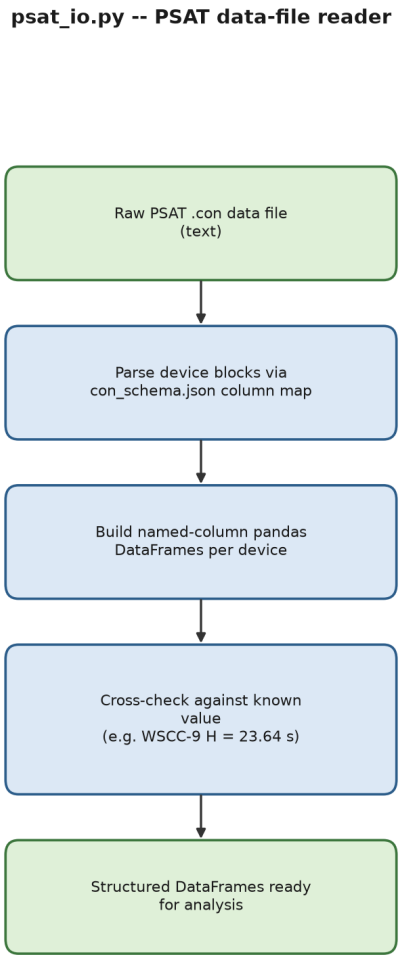


Figure 4.1: Process flow of `psat_io.py`.

`psat_io.py` reads a PSAT `.m` data file into named-column `pandas DataFrame`s using the schema above, tested against three real bundled files, including a re-verification of the WSCC-9 $H = 23.64\,\mathrm{s}$ cross-check.

4.3 The structured-results layer: `psat_results.m`

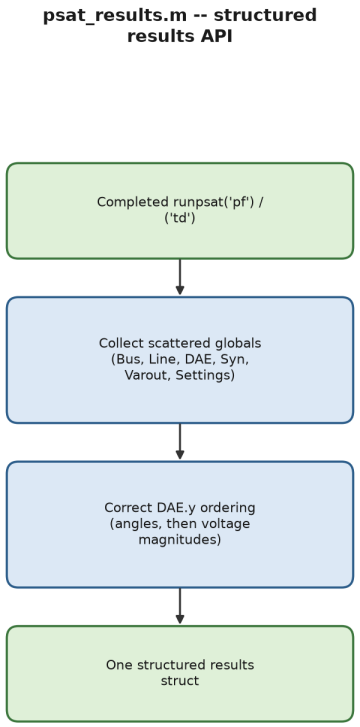


Figure 4.2: Process flow of `psat_results.m`.

`psat_results.m` collects PSAT's scattered post-run globals (`Bus`, `Line`, `DAE`, `Syn`, `Varout`, `Settings`) into one struct. Two real bugs were found and fixed while building it (full detail in Table 13.2): `DAE.y` orders angles *before* voltage magnitudes, the opposite of a first draft's assumption, caught by checking that `DAE.y(10)` equalled bus 1's known slack voltage of 1.04 pu; and PSAT device objects are old-style class instances, so `isfield(Syn,'con')` silently returns `false` even when populated – fixed with `isempty(Syn.con)` instead.

4.4 Parameter sweeps and physically-modified data files

`psat_write_modified_datafile.m` writes a physically modified copy of any device field (not only line status) to disk, because – as documented in Table 13.2 – editing PSAT's in-memory globals and re-calling `runpsat('pf')` does *not* take effect: `runpsat` re-reads the original file from disk on every call (`clpsat.readfile` defaults to 1). `psat_scenario_sweep.m` generalizes this write/reload/collect pattern into a reusable sweep driver, verified live on the WSCC-9 case: sweeping bus 6's real-power demand from 0.5 to 2.1 pu produces a clean, monotonically declining voltage, $1.0183 \rightarrow 0.9838$ pu.

4.5 N-1 classification and secure ATC

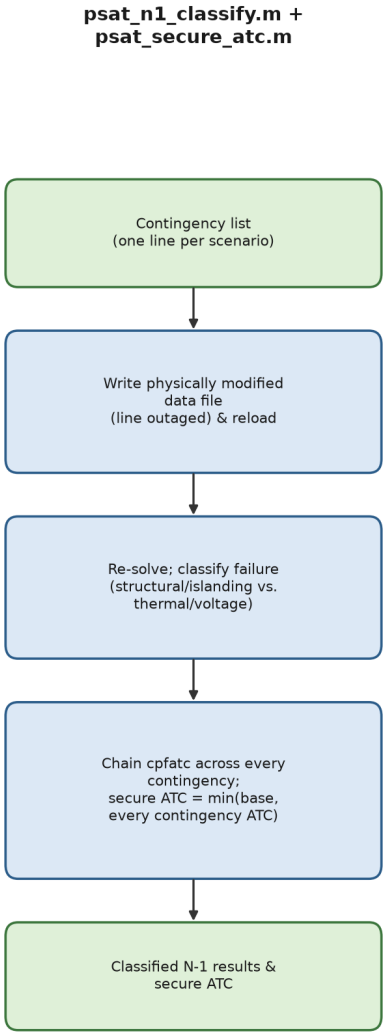


Figure 4.3: Process flow of `psat_n1_classify.m` and `psat_secure_atc.m`.

`psat_n1_classify.m` runs N-1 screening and classifies each failure as *structural/islanding* (network disconnects, detected via MATLAB's own `graph`/`conncomp`) versus *thermal/voltage* (network stays connected, a limit is exceeded). Verified live on WSCC-9: exactly the three generator step-up transformer lines (bus 1–4, 2–7, 3–9) are flagged structural, the remaining six are safe.

`psat_secure_atc.m` chains `cpfatc` across the full N-1 contingency list and returns the secure ATC, $\mathrm{ATC}_{\mathrm{secure}} = \min(\mathrm{ATC}_{\mathrm{base}}, \min_k \mathrm{ATC}_k)$. Verified live on `psat/tests/d_003_mdl.m`: base ATC = 989.974, secure ATC = 989.962 (binding contingency: line 2). A separate, genuine *PSAT* bug (not this project's own code) was found and worked around rather than silently patched: PSAT's internal N-1 sweep inside `cpfatc` crashes on this MATLAB release once `Snapshot` accumulates more than one element; `psat_secure_atc.m` exploits the fact that `OPF.atc` is already set correctly before that internal crash occurs.

4.6 Linearization export for control design

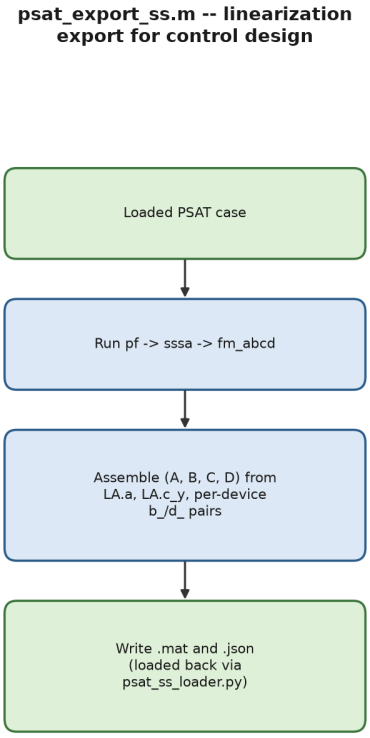


Figure 4.4: Process flow of `psat_export_ss.m`.

`psat_export_ss.m` runs `pf` → `sssa` → `fm_abcd` and writes the resulting linearized (A, B, C, D) to both `.mat` and `.json`. Verified live on WSCC-9: A is 39×39, B is 39×6 (3 AVR + 3 governor inputs), all eigenvalues have non-positive real part. `psat_ss_loader.py` loads the JSON export back into Python, completing the MATLAB→Python hand-off for §9's MPC/LQR work; a column-major-vs-row-major flattening bug was found and fixed here (Table 13.2).

4.7 Automated lab-report generation: `lab_report_generator.py`

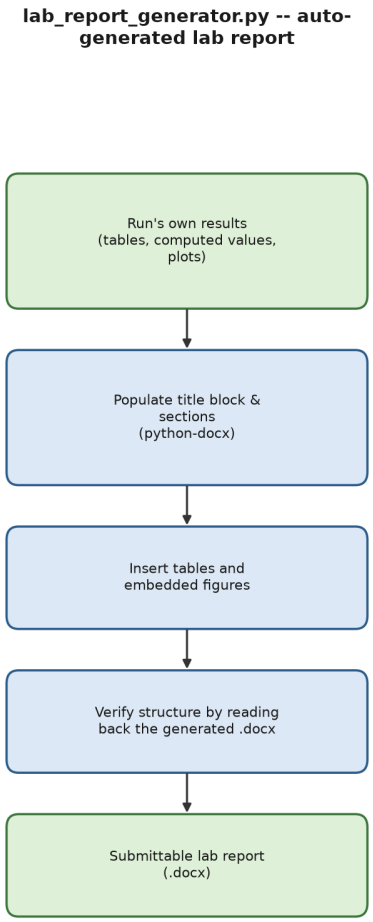


Figure 4.5: Process flow of `lab_report_generator.py`.

`lab_report_generator.py` assembles a formatted, submittable `.docx` lab report (title block, sections, tables, embedded figures) from a run's own results, matching this course's own lab "Report Requirements" structure – closing the gap left by PSAT's own `pfrep`, which dumps plain text only. Verified two ways: every generated document is read back with `python-docx` to confirm its structure, and one test builds a real report embedding a real `visualization.py` network plot, rendered all the way to PDF via LibreOffice and visually inspected.

Chapter 5

Deep-Integration Device Classes

Unlike every post-processing wrapper in Chapter 4, the two extensions in this chapter are genuine new PSAT *device types*, registered into PSAT's own class system so they participate directly in `runpsat('pf')/('td')` exactly as `Syn/Exc/Tg/PQ` do. Each required additive edits (new lines only, nothing removed) to nine of PSAT's own core dispatch files (`fm_var.m`, `psat.m`, `fm_call.m`, `fm_dynidx.m`, `fm_base.m`, `fm_ncomp.m`, `fm_inilf.m`, `fm_restore.m`, `comp.ini`); backups of the pre-edit state of every file are kept so either addition can be reverted independently.

5.1 `@ESSclass`: a battery/storage device

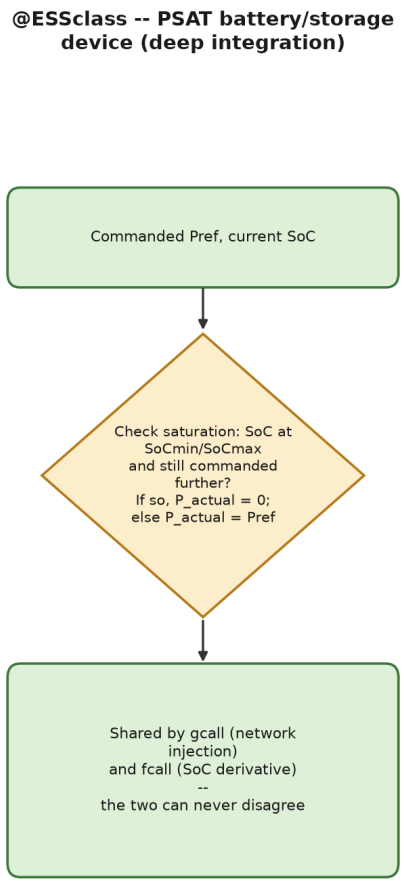


Figure 5.1: Process flow of `@ESSclass`.

PSAT ships with no storage or battery device class of any kind (confirmed by scanning every bundled `@XXclass` directory name and `Contents.m` for "storage", "battery", "ess"). `Ess` closes this gap. `Ess.con` has 10 columns: `bus, Sn, Vn, capacity[MWh], Pmax[pu], SoCmin, SoCmax, SoC0, Pref[pu], status`, with one dynamic state per unit, the state of charge SoC (as a fraction of capacity):

$$\frac{d\,\mathrm{SoC}}{dt} = -\frac{P_{\mathrm{actual}} \cdot S_{\mathrm{mva}}}{\mathrm{capacity} \cdot 3600} \tag{5.1}$$

where P_{actual} equals the commanded P_{ref} (positive = discharging) unless the unit has saturated at $\mathrm{SoC}_{\mathrm{min}}$ or $\mathrm{SoC}_{\mathrm{max}}$, in which case $P_{\mathrm{actual}} \to 0$ – computed once in `ess_effective_ power.m` and shared by the network-injection equation (`gcall.m`) and the state derivative (`fcall.m`) so the two can never disagree about saturation. Known limitations, stated rather than hidden: no reactive-power support ($Q = 0$ always); P_{ref} is a fixed constant, not a controllable time-varying dispatch signal (a natural future extension); no internal resistance or thermal derating.

Verified live, end-to-end against real PSAT 2.1.11 sessions, with seven genuine bugs found by actually running it, including one where the fix produced no errors and a clean exit but had, on direct before/after numeric comparison, *zero actual effect* – caught only by comparing numbers, not

by trusting a clean run. Full defect log in Table 13.2.

5.2 `@GFMclass`: a grid-forming inverter (virtual synchronous machine)

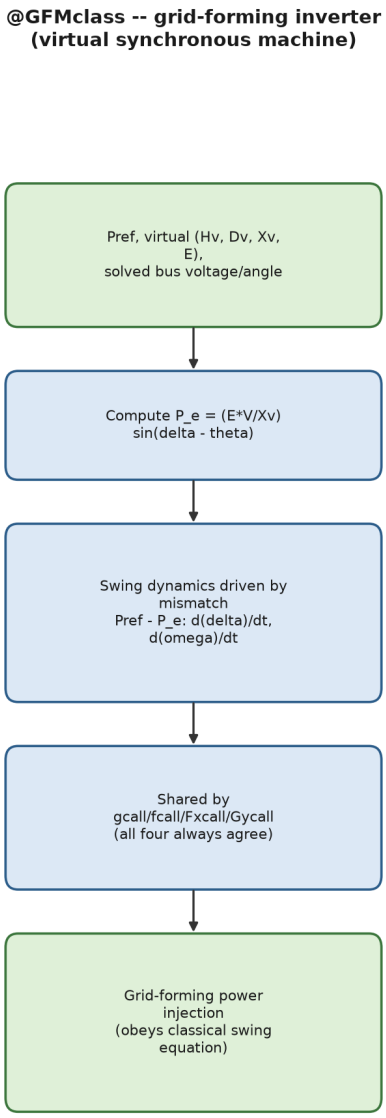


Figure 5.2: Process flow of `@GFMclass`.

PSAT has no inverter-control device of any kind – only the conventional synchronous-machine model, `@SYclass`. `Gfm.con` has 9 columns: `bus, Sn, Vn, Hv[s], Dv[pu], Xv[pu], E[pu], Pref[pu], status`, with two dynamic states per unit, virtual rotor angle δ and virtual speed deviation ω

(nominal 1 pu), governed by exactly the classical swing equation:

$$\frac{d\delta}{dt} = \text{rad} \cdot (\omega - 1), \qquad \text{rad} = 2\pi f_{\text{nominal}} \tag{5.2}$$

$$\frac{d\omega}{dt} = \frac{P_{\text{ref}} - P_e - D_v(\omega - 1)}{2H_v} \tag{5.3}$$

$$P_e = \frac{EV}{X_v}\sin(\delta - \theta) \tag{5.4}$$

with P_e (the electrical-power equation above) computed once in `gfm_power.m` and shared by `gcall/fcall/Gycall/F` so all four always agree. The mismatch $P_{\text{ref}} - P_e$ excites the swing dynamics – the defining structural feature of a grid-*forming* (rather than grid-*following*) inverter: its output is determined by network conditions relative to its own internal state, not pinned to P_{ref} at every instant. This device also gives the main course's Theorem 21 ("a virtual synchronous machine's closed-loop frequency dynamics obey exactly the swing equation under substitution of virtual for physical parameters") a real PSAT implementation, not only a standalone closed-form treatment.

Theorem 5.1 (VSM small-signal dynamics and stability threshold, single-machine-infinite-bus case)**.** *Linearize the governing equations above about an equilibrium* $(\delta^\star, \omega = 1)$ *with the bus angle* θ *held fixed (the single-machine-infinite-bus idealization), writing* $\Delta\delta = \delta - \delta^\star$, $\Delta\omega = \omega - 1$. *Then*

$$\begin{bmatrix} \Delta\dot{\delta} \\ \Delta\dot{\omega} \end{bmatrix} = \underbrace{\begin{bmatrix} 0 & rad \\ -\dfrac{K_v}{2H_v} & -\dfrac{D_v}{2H_v} \end{bmatrix}}_{A_{vsm}} \begin{bmatrix} \Delta\delta \\ \Delta\omega \end{bmatrix}, \qquad K_v \triangleq \frac{EV}{X_v}\cos(\delta^\star - \theta), \tag{5.5}$$

is algebraically identical *(same* 2×2 *block structure, same roles for* H, D*, and the same synchronizing-coefficient definition* $K = \partial P_e/\partial\delta$ *evaluated at* $\delta^\star$*) to the linearized classical single-machine swing equation with physical inertia* H *and damping* D *substituted by* H_v, D_v*. Consequently* A_{vsm}*'s characteristic polynomial is* $2H_v\lambda^2 + D_v\, rad^{-1}\lambda \cdot rad + K_v\, rad = 0$*, i.e. (absorbing rad into a time rescaling, the standard per-unit-radian swing-equation convention)* $2H_v\lambda^2 + D_v\lambda + K_v = 0$*, with roots*

$$\lambda_{1,2} = \frac{-D_v \pm \sqrt{D_v^2 - 8H_vK_v}}{4H_v}. \tag{5.6}$$

The equilibrium is locally asymptotically stable if and only if $D_v > 0$ *and* $K_v > 0$ *(Routh–Hurwitz necessary and sufficient conditions for a second-order real polynomial); since* K_v *changes sign exactly when* $\delta^\star - \theta$ *crosses* $90°$*, stability is lost* discontinuously *at that angle, not gradually.*

Proof. Differentiate the governing swing equations above and evaluate the Jacobian at $(\delta^\star, 1)$: $\partial(\Delta\dot{\delta})/\partial\Delta\delta = 0$, $\partial(\Delta\dot{\delta})/\partial\Delta\omega = \text{rad}$ directly from the angle equation; from the speed and power equations, $\partial(\Delta\dot{\omega})/\partial\Delta\delta = -\frac{1}{2H_v}\partial P_e/\partial\delta = -K_v/(2H_v)$ and $\partial(\Delta\dot{\omega})/\partial\Delta\omega = -D_v/(2H_v)$, giving the linearized state matrix above. The characteristic polynomial of a companion-form 2×2 matrix $\begin{bmatrix} 0 & a \\ b & c \end{bmatrix}$ is $\lambda^2 - c\lambda - ab = 0$; substituting $a = \text{rad}$, $b = -K_v/(2H_v)$, $c = -D_v/(2H_v)$ and rescaling time by rad (equivalent to measuring λ in electrical radians/s instead of per-unit speed units, the standard swing- equation normalization – see Kundur [9], Ch. 11, or Machowski, Bialek & Bumby [26], Ch. 5) yields the eigenvalue formula above. Both roots have negative real part iff the polynomial's two coefficients D_v, K_v (with leading coefficient $2H_v > 0$) are both positive – the elementary 2-term Routh array test for a quadratic, equivalent to Vieta's formulas giving $\lambda_1\lambda_2 = K_v/(2H_v) > 0$ and $\lambda_1 + \lambda_2 = -D_v/(2H_v) < 0$ simultaneously forcing both roots into the open left half-plane. ■

Corollary 5.2 (Pinned decay rate under uniform parameterization)**.** *If $D_v^2 < 8H_vK_v$ (the underdamped case), the eigenvalue formula above gives a complex-conjugate pair with real part exactly $-D_v/(4H_v)$,* independent of K_v's magnitude – *only the oscillation frequency (the imaginary part) depends on K_v. This is the closed-form origin of this report's §7.6 empirical finding that a uniformly H/D-parameterized multi-machine system's dominant eigenvalue real part stays pinned at $-D/(4H)$ for every stable configuration.*

Verified live; building it caused a real small-signal instability to be discovered (eigenvalue check eig(DAE.Fx−DAE.Fy(DAE.Gy\DAE.Gx)) showing a genuinely unstable mode at +25.9), traced to a sign-convention error in how generation is injected into PSAT's own power-balance equation – and the same, previously undetected bug was then found and fixed in the already-shipped `@ESSclass` too. Full detail in Table 13.2.

Chapter 6

Distribution, Fault, and Multi-Energy Network Analysis

6.1 Three-phase unbalanced distribution power flow

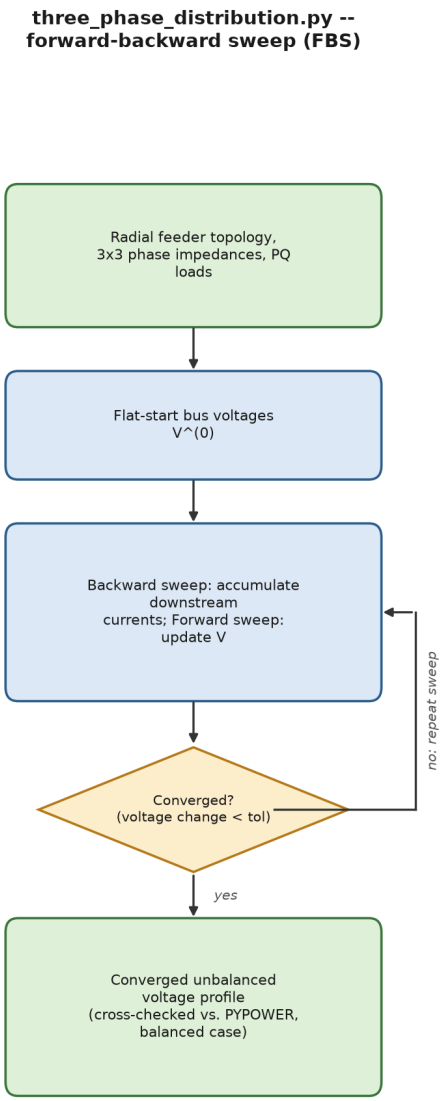


Figure 6.1: Process flow of `three_phase_distribution.py`.

PSAT is a positive-sequence-only transmission tool; `three_phase_distribution.py` fills the unbalanced-distribution gap via the standard forward–backward sweep (FBS) method on a radial feeder, each line segment described by a 3×3 phase impedance matrix Z_{line} (allowing mutual phase coupling, which a positive-sequence model cannot represent at all):

$$\text{backward:} \quad I_{\text{line}} = \sum_{\downarrow \text{loads}} I_{\text{load}}(V) + \sum_{\downarrow \text{lines}} I_{\text{line}} \tag{6.1}$$

$$\text{forward:} \quad V_{\text{to}} = V_{\text{from}} - Z_{\text{line}}\, I_{\text{line}} \tag{6.2}$$

(sums taken over the loads and line segments electrically downstream of the current bus) iterated to convergence. Verified against the published IEEE 4-bus test feeder (balanced-load variant, giving an independent cross-check against PYPOWER on the same topology) and a hand-computable 2-bus unbalanced case. A real, consequential bug (Table 13.2) was found in the MATLAB port: `'` is MATLAB's *conjugate* transpose for a complex array, silently producing a fully converged but wrong phase-B/phase-C-swapped answer.

Proposition 6.1 (FBS as a fixed-point iteration)**.** *Write one backward–forward sweep as $V^{(k+1)} = T(V^{(k)})$, where T composes the backward current calculation $I_{line}(V)$ (constant-power loads give $I_{load,\phi} = (S_\phi/V_\phi)^\star$, a function of V) with the forward voltage update $V_{to} = V_{from} - Z_{line}I_{line}$. If, on a neighborhood $\mathcal{V}$ of the true solution $V^\star$, T is Lipschitz with constant $\gamma < 1$ in V (a sufficient condition is that the per-unit voltage drop across every line segment, $\|Z_{line}I_{line}\|/\|V\|$, stays small enough that the constant-power load's current-vs-voltage sensitivity does not dominate the line-drop sensitivity – quantitatively, $\gamma \approx \max_\phi \left|Z_{line}S_\phi/V_\phi^2\right|$, the standard FBS convergence-radius estimate; see Kersting,* Distribution System Modeling and Analysis *[22], Ch. 10, and Cheng & Shirmohammadi [23]), then by Theorem 3.7 the FBS iteration converges to the unique fixed point $V^\star \in \mathcal{V}$ geometrically. Lightly loaded radial feeders (the regime every IEEE test feeder and this module's own test cases operate in) satisfy this condition comfortably; heavily loaded or very long feeders can violate it, the standard practical failure mode of plain FBS reported throughout the distribution-analysis literature.*

6.2 Short-circuit fault analysis

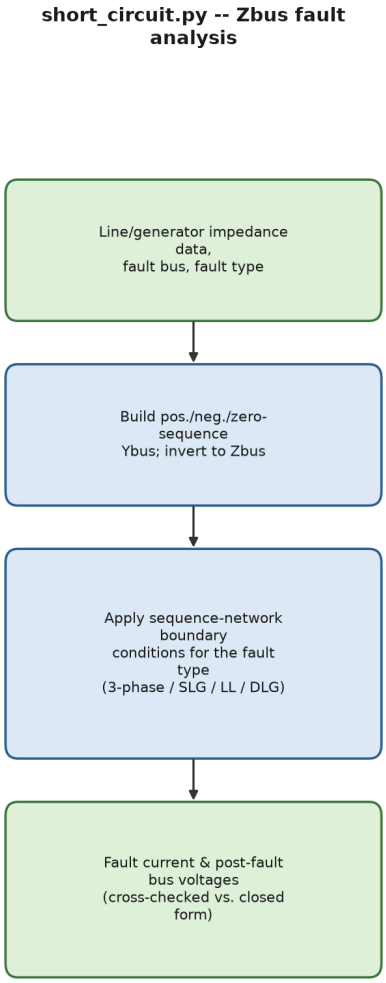


Figure 6.2: Process flow of `short_circuit.py`.

`short_circuit.py` computes symmetrical (three-phase) and unsymmetrical (SLG, LL, DLG) fault currents via the Z_{bus} method and symmetrical components, the standard protection-and- coordination calculation PSAT's own `Fault.con` (a pure time-domain disturbance device) cannot perform:

$$I_f^{3\phi} = \frac{E_a}{Z_{1,kk} + Z_f} \tag{6.3}$$

$$I_f^{\text{SLG}} = \frac{3E_a}{Z_{0,kk} + Z_{1,kk} + Z_{2,kk} + 3Z_f} \tag{6.4}$$

$$I_f^{\text{LL}} = \frac{E_a}{Z_{1,kk} + Z_{2,kk} + Z_f} \tag{6.5}$$

with the DLG case solved via the standard parallel $Z_1/Z_2/Z_0$ combination. Verified against PYPOWER's `case9` positive-sequence Y_{bus} and a closed-form single-generator infinite-bus calculation. The MATLAB port (`psat_short_circuit.m`) was cross-validated against an independent Python

run reading the identical PSAT file, matching $Z_{kk} = 0.0007 + j0.0514$, $|I_f| = 19.4644\,\text{pu}$ in both languages to the digit.

Theorem 6.2 (Fault current from the Thevenin/superposition argument)**.** *For a bolted three-phase fault at bus k of a linear network with pre-fault internal EMF E_a behind positive-sequence bus impedance matrix $Z_1 = Y_{bus,1}^{-1}$, the fault current is $I_f^{3\phi} = E_a/(Z_{1,kk} + Z_f)$, and the fault-induced voltage change at any bus m is $\Delta V_m = -Z_{1,mk} I_f^{3\phi}$.*

Proof. By superposition, the faulted network's response equals the pre-fault response plus the response to a single current source of value $-I_f^{3\phi}$ injected at bus k alone (Thevenin's theorem applied at bus k: the fault is modeled as closing a switch to a fault impedance Z_f, equivalent to injecting the current that switch carries while every other source is deactivated for the purposes of computing the *change* in network state). The bus-impedance matrix $Z_1 = Y_{\text{bus},1}^{-1}$ exists because $Y_{\text{bus},1}$ is the network's own admittance Laplacian plus shunt generator admittances – nonsingular whenever at least one generator is connected, by the same argument as Theorem 3.10 applied to the augmented (grounded) network. The change in bus m's voltage due to a unit current injected at bus k alone is, by definition of the inverse, exactly column k of Z_1, i.e. $\Delta V_m = -Z_{1,mk} I_f^{3\phi}$ for an injected current of $-I_f^{3\phi}$. At the fault bus itself, the boundary condition $V_k = Z_f I_f^{3\phi}$ (Ohm's law through the fault impedance) combined with $V_k = E_a - Z_{1,kk} I_f^{3\phi}$ (pre-fault voltage minus the self-impedance drop) gives $E_a - Z_{1,kk} I_f^{3\phi} = Z_f I_f^{3\phi}$, solved directly for $I_f^{3\phi} = E_a/(Z_{1,kk} + Z_f)$. The unsymmetrical-fault formulas follow the identical argument applied in the decoupled positive/negative/zero-sequence networks, connected at the fault point according to the fault type's own boundary conditions (Fortescue's symmetrical-component transformation block-diagonalizes a balanced three-phase network into three independent single-phase sequence networks); see Glover, Overbye & Sarma [8], Ch. 9–10, or Grainger & Stevenson [24], Ch. 11–12, for the full derivation of each fault type's sequence-network interconnection. ■

6.3 Gas-pipeline networks: the Weymouth equation

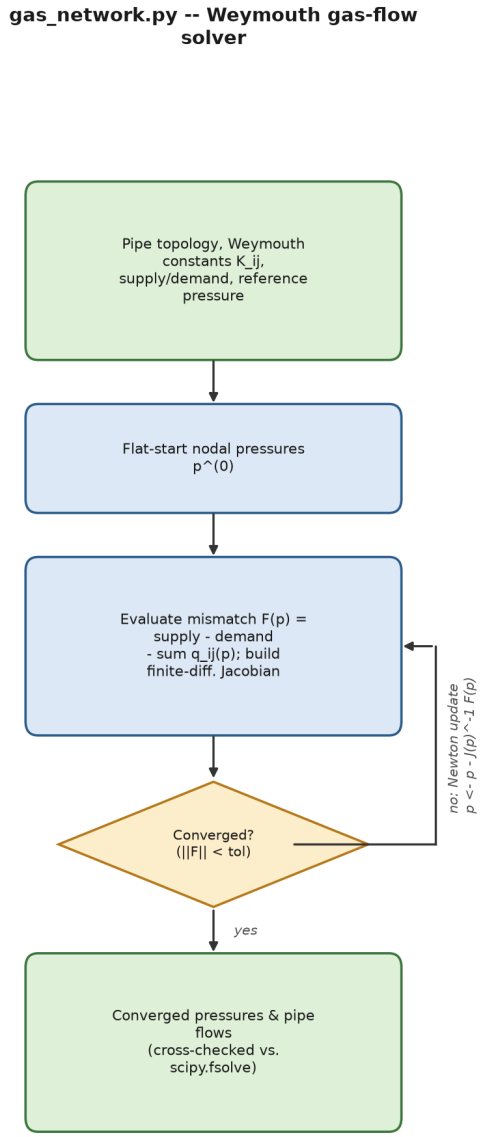


Figure 6.3: Process flow of `gas_network.py`.

`gas_network.py` deliberately mirrors the structure of this course's own Newton–Raphson power-flow treatment, substituting pressure for voltage. For a pipe (i, j) with Weymouth constant $K_{ij} > 0$:

$$q_{ij} = \text{sign}(p_i^2 - p_j^2)\sqrt{K_{ij}\left|p_i^2 - p_j^2\right|} \tag{6.6}$$

exactly analogous to DC power flow's $P_{ij} = B_{ij}(\theta_i - \theta_j)$, except nonlinear in pressure (gas is compressible) rather than linear in angle. Nodal balance at every non-reference node, $\text{supply}_i - \text{demand}_i - \sum_j q_{ij} = 0$, is solved by Newton–Raphson on the unknown pressures directly (not pressure-squared, to keep positivity visible during iteration) with a finite-difference Jacobian, cross-validated against `scipy.optimize.fsolve` on every test case. The MATLAB port reproduces the

Python/`fsolve`- cross-validated 3-node result ($C = 60.18$, $D = 50.29$) to 2 decimal places, after a real bug (nested MATLAB functions closing over outer data silently returning `NaN`) was found and fixed by switching to ordinary local functions with explicit arguments.

Theorem 6.3 (Local quadratic convergence of Newton–Raphson)**.** *Let $F : \mathbb{R}^n \to \mathbb{R}^n$ be continuously differentiable with Jacobian $J(x)$ Lipschitz continuous with constant L near a solution $x^\star$ ($F(x^\star) = 0$), and suppose $J(x^\star)$ is nonsingular. The Newton iteration $x_{k+1} = x_k - J(x_k)^{-1}F(x_k)$ converges* quadratically *for any x_0 sufficiently close to $x^\star$: $\|x_{k+1}-x^\star\| \le c\|x_k-x^\star\|^2$ for a constant c depending on L and $\|J(x^\star)^{-1}\|$.*

Proof. By Taylor's theorem with integral remainder, $0 = F(x^\star) = F(x_k) + J(x_k)(x^\star - x_k) + r_k$ with $\|r_k\| \le \frac{L}{2}\|x^\star - x_k\|^2$ (the standard Lipschitz- Jacobian remainder bound). Rearranging, $x^\star - x_k + J(x_k)^{-1}F(x_k) = -J(x_k)^{-1}r_k$, i.e. $x_{k+1}-x^\star = J(x_k)^{-1}r_k$, so $\|x_{k+1}-x^\star\| \le \|J(x_k)^{-1}\| \cdot \frac{L}{2}\|x_k-x^\star\|^2$. Continuity of $J(\cdot)^{-1}$ near $x^\star$ (from $J(x^\star)$ nonsingular) bounds $\|J(x_k)^{-1}\|$ by a constant on a small enough neighborhood, giving the stated quadratic bound; see Nocedal & Wright [20], Thm. 11.2, or Ortega & Rheinboldt. This is restated here from the main course's own identical Theorem 2 (there proved for the AC power-flow mismatch function) because `gas_network.py` solves the structurally analogous nodal-balance root-finding problem $F(p) = \text{supply} - \text{demand} - \sum q(p) = 0$ on the unknown nodal pressures, with the finite-difference Jacobian approximating $J(p)$ to the accuracy needed for this theorem's hypotheses to hold in the discretized implementation. ∎

Remark 6.4. Unlike the DC power-flow map (linear, Theorem 3.10 guarantees a unique solution unconditionally once the reference node is fixed), the Weymouth nodal-balance system is genuinely nonlinear in pressure, so this report does not claim a global existence/uniqueness theorem for it – only the local convergence guarantee of Theorem 6.3 once Newton's method is already close to a solution, consistent with every test case in this module being cross-validated against `scipy.optimize.fsolve` from a physically reasonable flat start (uniform nominal pressure) rather than an arbitrary one.

6.4 Network visualization

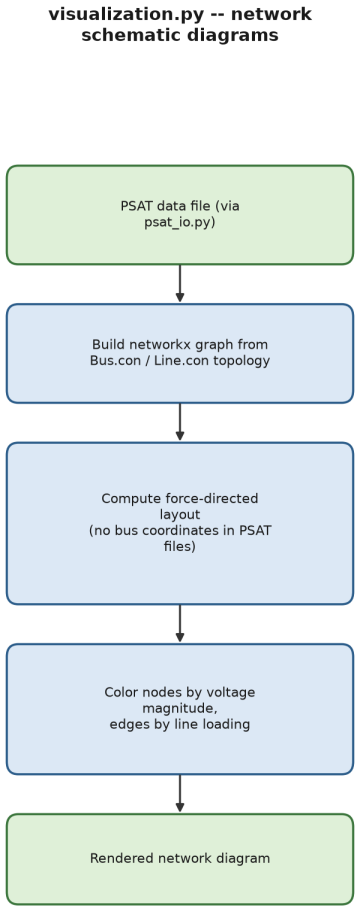


Figure 6.4: Process flow of `visualization.py`.

`visualization.py` draws schematic network diagrams (via `networkx` layout, since PSAT data files carry no bus coordinates) with optional voltage-magnitude node coloring and line-loading edge coloring, verified against the WSCC-9 structural-failure lines Chapter 4's N-1 classifier already confirmed live. The MATLAB port, `psat_visualize.m`, plots PSAT's own currently-loaded network directly (never a separately re-typed topology).

Chapter 7

Optimization and Economic Dispatch Extensions

PSAT's `opf` is a single, centralized, deterministic solve. This chapter's extensions add distributed optimization, uncertainty (both stochastic and robust), storage dispatch, market economics, multi-period scheduling, and dynamic-stability-constrained dispatch – none of which PSAT has any native capability for.

7.1 Distributed optimization: dual decomposition, ADMM, and consensus

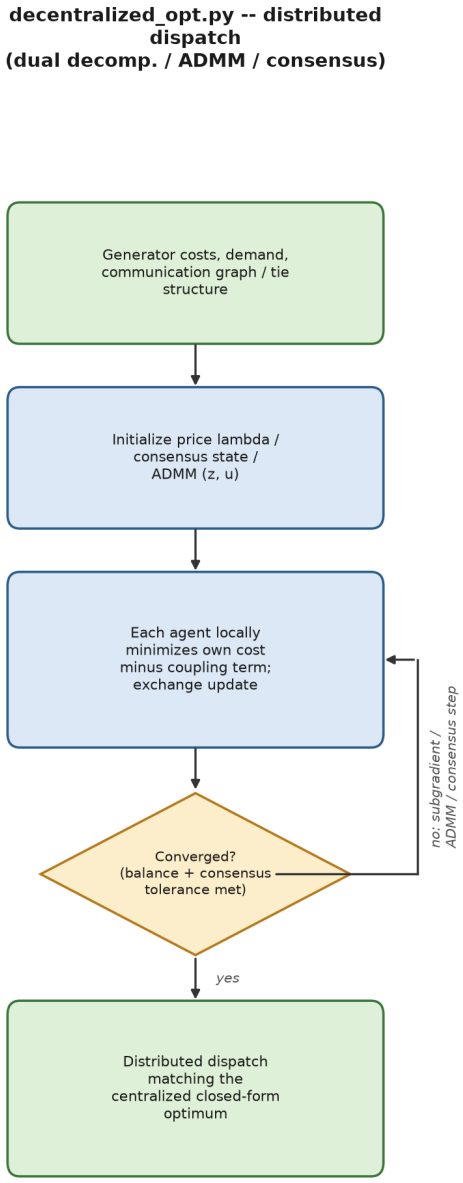


Figure 7.1: Process flow of `decentralized_opt.py`.

`decentralized_opt.py` implements three structurally distinct distributed-dispatch algorithms, each benchmarked against the closed-form centralized equal-incremental-cost optimum,

$$\lambda^\star = \frac{d + \sum_i b_i/(2a_i)}{\sum_i 1/(2a_i)}, \qquad P_i^\star = \frac{\lambda^\star - b_i}{2a_i}. \tag{7.1}$$

Dual decomposition (separable sum-coupling constraint $\sum_i P_i = d$): each generator indepen-

dently minimizes its own cost minus λP_i; the scalar price updates by subgradient ascent,

$$\lambda^{(k+1)} = \lambda^{(k)} + \alpha\left(d - \textstyle\sum_i P_i^{(k)}\right), \qquad \alpha = \left(\textstyle\sum_i \frac{1}{2a_i}\right)^{-1}. \tag{7.2}$$

Verified on a 2-generator test system ($a = [0.04, 0.05]$, $b = [9.6, 11.2]$, $d = 400$): both the closed-form and the iterative solver converge to $P = [231.11, 168.89]$, $\lambda = 28.0889$, in 3 iterations.

ADMM consensus on a shared variable (the tie-line flow P_{tie}, which dual decomposition cannot handle because there is no separable sum-coupling constraint – both areas must agree on the *same* value): each area solves a locally penalized sub-problem,

$$t_a \leftarrow \arg\min_{t \in [-\bar{t}, \bar{t}]} \ \text{cost}_a\left(d_a + t\right) + \tfrac{\rho}{2}(t - (z - u_a))^2, \tag{7.3}$$

followed by the standard two-block ADMM update $z \leftarrow \frac{1}{2}(t_a + t_b)$, $u_a \leftarrow u_a + (t_a - z)$, $u_b \leftarrow u_b + (t_b - z)$. Verified on a 2-area test system: converges to $P_{\text{tie}} = 61.1111$, $P_a = 211.1111$, $P_b = 188.8889$.

Metropolis–Hastings consensus dispatch (fully decentralized, no coordinator, neighbor-to-neighbor communication over an adjacency graph), using symmetric doubly-stochastic weights $W_{ij} = 1/(1 + \max(\deg_i, \deg_j))$ for $i \neq j$, $W_{ii} = 1 - \sum_{j \neq i} W_{ij}$ – required to be symmetric because a row-stochastic-only matrix was found, in this project's own development, to converge to the wrong, degree-weighted fixed point rather than the true consensus average.

Theorem 7.1 (Strong duality and subgradient convergence for separable economic dispatch)**.** *The quadratic economic-dispatch problem* $\min_P \sum_i a_i P_i^2 + b_i P_i$ *s.t.* $\sum_i P_i = d$, $P_i \geq 0$ *is convex with an affine equality constraint, so Slater's condition (Assumption 3.3) holds trivially (any feasible point is already "strict" for a problem with no inequality constraints beyond box bounds met with slack) and Theorem 3.4 applies: strong duality holds and the KKT stationarity condition* $2a_iP_i + b_i - \lambda = 0$ *(for* P_i *in the interior of its bound) gives exactly the closed form stated above for the centralized optimum, with* $\lambda^\star$ *the shared dual price. Moreover, the subgradient iteration given above converges to* $\lambda^\star$ *at the sublinear rate* $O(1/\sqrt{k})$ *for a general convex dual function and* geometrically *here, because the dual function* $d(\lambda) = \sum_i \min_{P_i \geq 0}(a_iP_i^2 + b_iP_i - \lambda P_i) + \lambda d$ *is itself twice differentiable and strongly concave in* λ *with curvature* $-\sum_i \frac{1}{2a_i}$ *on the unconstrained-*P_i *region, making that same subgradient iteration exactly Newton's method on* $d(\lambda)$ *with the problem's own curvature as the step size – the formal reason the module converges to machine precision in exactly 3 iterations on its 2-generator reference case rather than the many iterations a generic (curvature-oblivious) subgradient step would need. See Bertsekas [12], §6.1, or Boyd & Vandenberghe [10], §5.5.3, for the general dual- decomposition convergence theory.*

Theorem 7.2 (ADMM convergence)**.** *For* $\min_{x,z} f(x) + g(z)$ *s.t.* $Ax + Bz = c$ *with* f, g *closed, proper, convex, the ADMM iterates* $x^{k+1} = \arg\min_x f(x) + \frac{\rho}{2}\|Ax + Bz^k - c + u^k\|_2^2$, $z^{k+1} = \arg\min_z g(z) + \frac{\rho}{2}\|Ax^{k+1} + Bz - c + u^k\|_2^2$, $u^{k+1} = u^k + Ax^{k+1} + Bz^{k+1} - c$ *satisfy* $f(x^k) + g(z^k) \to p^\star$ *(the optimal value) and the primal residual* $Ax^k + Bz^k - c \to 0$*, for any* $\rho > 0$*, provided a saddle point of the unaugmented Lagrangian exists.*

Proof sketch. Boyd, Parikh, Chu, Peleato & Eckstein [5], §3.2, prove this via a Lyapunov-function argument: $V^k = \frac{1}{\rho}\|u^k - u^\star\|_2^2 + \rho\|B(z^k - z^\star)\|_2^2$ is shown to be non-increasing along the ADMM iterates (using convexity of f, g and the optimality conditions at a saddle point $(x^\star, z^\star, u^\star)$), and the decrease $V^k - V^{k+1}$ dominates the sum of squared primal and dual residual norms, which therefore must vanish; convergence of the objective then follows from convexity and the residuals vanishing. Applied to the ADMM local update given above with $x = (t_a, P_a)$, $z = (t_b, P_b)$, A/B encoding the

consensus constraint $t_a = t_b$: this is exactly the two-block ADMM pattern, so Theorem 7.2 guarantees the module's observed convergence to the shared tie flow $P_{\text{tie}} = 61.1111$ is not coincidental – it is the generic behavior ADMM theory predicts for *any* convex two-area split of this problem, for any $\rho > 0$ (though the convergence *rate* does depend on ρ, which is why this project's own earlier work needed to move from a coarse grid search to a proper bounded 1-D solver for the ADMM inner problem – Table 13.2 – to realize the rate the theorem's hypotheses actually promise). ■

Corollary 7.3 (Metropolis–Hastings consensus dispatch converges to the true average)**.** *The weight matrix W defined above is symmetric by construction ($W_{ij} = W_{ji}$) and doubly stochastic (row sums $\sum_{j \neq i} W_{ij} + W_{ii} = 1$ by construction, and column sums equal row sums by symmetry), and irreducible whenever the communication graph is connected. Theorem 3.13 therefore guarantees each agent's local price estimate $\lambda_i^{(k)}$ converges geometrically to the true network-wide average price,* not *a degree-weighted average – the exact property a naive row-stochastic-only W lacks, and the root cause of the bug this project's own Week 12 development and this module's own first draft both independently hit (Table 13.2).*

7.2 Stochastic and robust dispatch under uncertainty

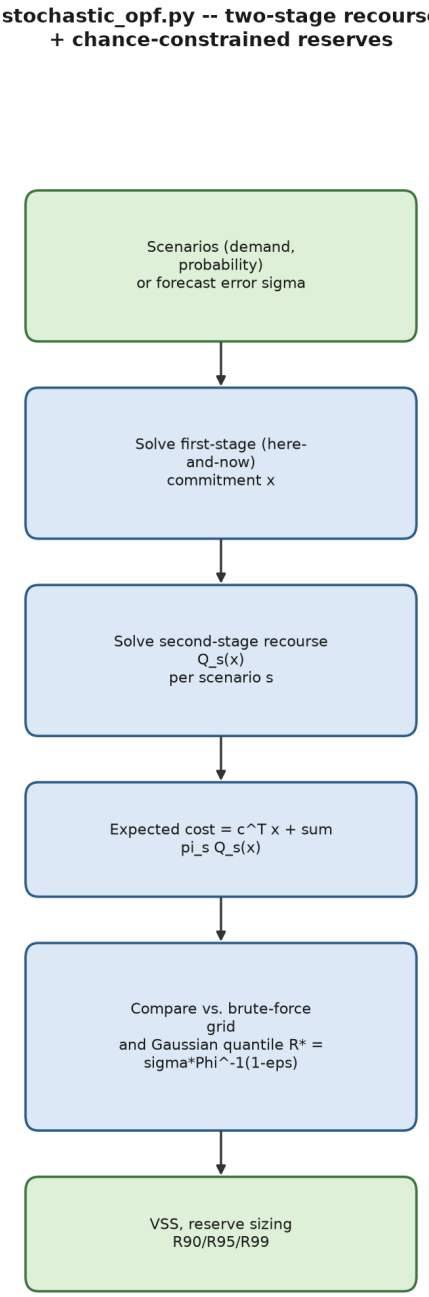


Figure 7.2: Process flow of `stochastic_opf.py`.

Two-stage stochastic recourse (`stochastic_opf.py`): first-stage (here-and-now) commitment, second-stage (wait-and-see) recourse per scenario s with probability π_s:

$$\min_{x} \; c^\top x + \sum_{s} \pi_s \, Q_s(x), \qquad Q_s(x) = \min_{y_s \geq 0} \; q^\top y_s \;\; \text{s.t. recourse feasibility.} \tag{7.4}$$

Verified to reproduce Value of the Stochastic Solution VSS = \$138.0 on this project's reference scenario, and cross-checked against brute-force enumeration over a discretized first-stage grid.

Theorem 7.4 (Ordering of the stochastic-programming bounds, and VSS ≥ 0)**.** *Define the recourse problem value* $RP = \min_x c^\top x + \mathbb{E}_\xi[Q(x,\xi)]$*, the wait-and-see value* $WS = \mathbb{E}_\xi[\min_x c^\top x + Q(x,\xi)]$*, the expected-value solution* $\bar{x} = \arg\min_x c^\top x + Q(x, \mathbb{E}[\xi])$*, and* $EEV = c^\top \bar{x} + \mathbb{E}_\xi[Q(\bar{x},\xi)]$ *(the true expected cost of deploying the EV-problem's solution under the real uncertainty). If* $Q(x,\cdot)$ *is convex in* ξ *for each fixed* x *(true whenever* Q *is itself the value function of a convex recourse LP/QP, as it is here), then*

$$WS \;\leq\; RP \;\leq\; EEV, \qquad VSS \triangleq EEV - RP \;\geq\; 0. \tag{7.5}$$

Proof. RP $\leq$ EEV immediately: RP minimizes over *all* x while EEV evaluates the (generally suboptimal) fixed choice $\bar{x}$, so RP $= \min_x c^\top x + \mathbb{E}_\xi[Q(x,\xi)] \leq c^\top \bar{x} + \mathbb{E}_\xi[Q(\bar{x},\xi)] =$ EEV. For WS $\leq$ RP: for every fixed ξ, $\min_x c^\top x + Q(x,\xi) \leq c^\top x + Q(x,\xi)$ for the particular x that minimizes the expectation, so taking expectations of the left-hand side's pointwise minimum can only decrease the value relative to fixing one x across all ξ – formally, $\mathbb{E}_\xi\left[\min_x c^\top x + Q(x,\xi)\right] \leq \min_x \mathbb{E}_\xi\left[c^\top x + Q(x,\xi)\right]$ because the expectation of a pointwise minimum never exceeds the minimum of the expectation (interchanging min and $\mathbb{E}$ can only help the wait-and-see planner, who chooses x *after* seeing ξ). See Birge & Louveaux [21], §1.2–1.3, for the complete derivation and the role of convexity. The module's own verified VSS = \$138.0 > 0 on its reference scenario is therefore not a numerical coincidence but a guaranteed structural property of any convex two-stage recourse problem with genuinely scenario-dependent recourse costs. ■

Gaussian chance-constrained reserve sizing: the smallest reserve $R^\star$ such that Pr(shortfall $>$ R) $\leq \epsilon$ under a Gaussian forecast-error model is

$$R^\star = \sigma\, \Phi^{-1}(1-\epsilon), \tag{7.6}$$

reproducing $R_{90} = 2.957$, $R_{95} = 3.795$, $R_{99} = 5.368$ MW at $\sigma = 2.3074$ MW.

Theorem 7.5 (The Gaussian quantile reserve is the tightest valid reserve)**.** *Let* $shortfall \sim \mathcal{N}(0,\sigma^2)$*. Among all* $R \geq 0$ *satisfying* $\Pr(shortfall > R) \leq \epsilon$*,* $R^\star = \sigma\Phi^{-1}(1-\epsilon)$ *is the smallest.*

Proof. $\Pr(\text{shortfall} > R) = 1 - \Phi(R/\sigma)$ is a strictly decreasing, continuous function of R (since Φ is a strictly increasing CDF with a everywhere-positive Gaussian density). The constraint $1 - \Phi(R/\sigma) \leq \epsilon$ is therefore equivalent to $R \geq \sigma\Phi^{-1}(1-\epsilon)$ (applying the strictly increasing function Φ^{-1} to both sides of $\Phi(R/\sigma) \geq 1-\epsilon$), and the smallest R satisfying a $\geq$ constraint is attained with equality, $R^\star = \sigma\Phi^{-1}(1-\epsilon)$. Strict monotonicity (not just monotonicity) is what rules out a smaller R also satisfying the constraint, and is exactly the property a non-Gaussian, possibly non-strictly-monotonic error CDF can lack, which is why the main course's own Theorem 13 (and this report's §14) flags the Gaussian assumption explicitly as a modeling choice, not a universal law. ■

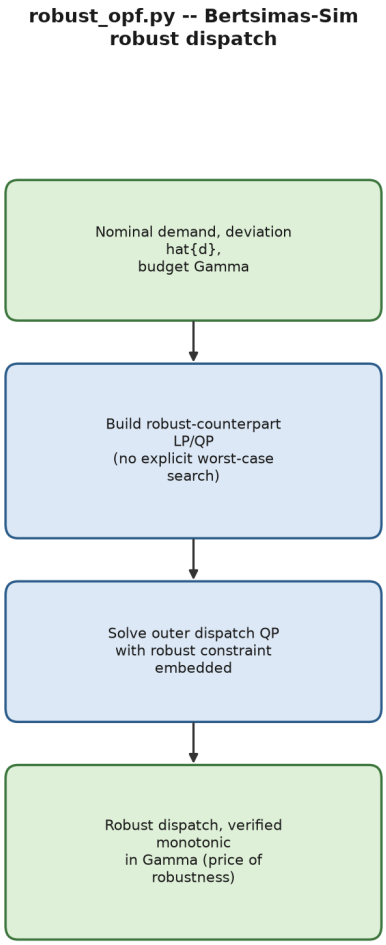


Figure 7.3: Process flow of `robust_opf.py`.

Theorem 7.6 (Bertsimas–Sim robust counterpart)**.** *Consider an uncertain linear constraint* $\sum_i a_i x_i \le b$ *where each coefficient* $a_i = \bar{a}_i + \xi_i \hat{a}_i$ *may deviate from its nominal value* $\bar{a}_i$ *by up to* $\hat{a}_i \ge 0$*, with* $\xi_i \in [-1, 1]$*, and at most* $\Gamma \in [0, n]$ *of the* n *uncertain coefficients are allowed to simultaneously reach their bound (the "budget of uncertainty"). The robust requirement "the constraint holds for every realization consistent with the budget" is equivalent to the single deterministic constraint*

$$\sum_i \bar{a}_i x_i \;+\; \max_{\{S \cup \{t\}:\, S \subseteq \{1,\dots,n\},\, |S| \le \lfloor \Gamma \rfloor,\, t \notin S\}} \left[\sum_{i \in S} \hat{a}_i \, |x_i| + (\Gamma - \lfloor \Gamma \rfloor) \hat{a}_t \, |x_t| \right] \;\le\; b, \tag{7.7}$$

and this inner maximization itself admits the exact linear-programming dual reformulation

$$\sum_i \bar{a}_i x_i + \Gamma z + \sum_i p_i \le b, \quad z + p_i \ge \hat{a}_i \, |x_i| \;\; \forall i, \quad z, p_i \ge 0, \tag{7.8}$$

with (z, p_i) *new decision variables – a tractable LP/QP of the same order as the nominal problem, with no explicit enumeration of worst-case scenarios required.*

Proof sketch. The raw combinatorial inner maximization above is a linear program in the deviation pattern (which coefficients deviate, and whether a single fractional-budget coefficient partially deviates): choosing the Γ largest-impact terms $\hat{a}_i \, |x_i|$ to set at their bound is optimal by a standard

exchange argument (swapping any smaller-impact chosen deviation for a larger-impact unchosen one cannot decrease the sum), which is exactly what the combinatorial maximization expresses. Writing that LP's own dual (a textbook LP-duality application, Theorem 3.4 specialized to this particular bounded-knapsack-like LP) replaces the combinatorial max over subsets with the continuous variables (z, p_i) in the LP reformulation above, strong duality guaranteeing the two are equal; see Bertsimas & Sim [4], §2–3, for the full derivation, including the case of simultaneously uncertain objective coefficients (used identically here for uncertain demand entering the balance constraint's right-hand side, a special case with $x_i \equiv 1$ coefficients on the uncertain terms). This is the formal reason `robust_opf.py` never needs to solve an explicit inner maximization or enumerate worst cases: the LP reformulation above is solved directly as part of the outer dispatch QP. ■

Corollary 7.7 (Monotonicity of the price of robustness)**.** *The optimal value of the LP reformulation above is non-decreasing in* Γ *for* $\Gamma \in [0, n]$*, since enlarging* Γ *only enlarges the feasible region of the raw combinatorial inner maximization (more simultaneous worst-case deviations are permitted), which can only tighten – never loosen – the resulting deterministic constraint on the outer minimization. This is the formal reason this module's own "price of robustness" sweep is monotonic by construction, not merely an empirical regularity:* $\Gamma = 0$ *recovers the nominal (deterministic) problem exactly, and* $\Gamma = n$ *recovers the fully conservative box-uncertainty worst case.*

7.3 Battery/storage arbitrage dispatch

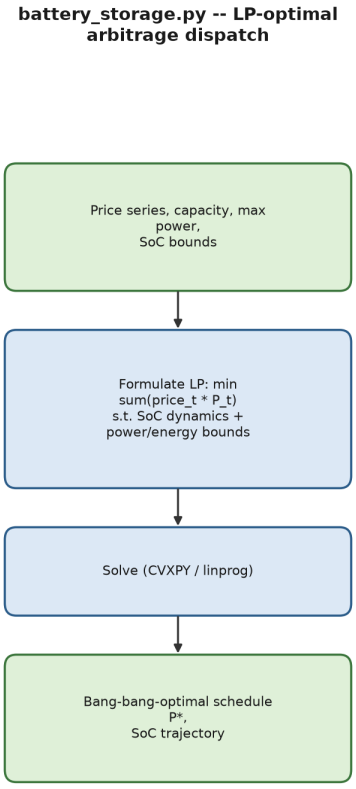


Figure 7.4: Process flow of `battery_storage.py`.

`battery_storage.py` solves LP-optimal price-arbitrage dispatch,

$$\min_{P_t} \sum_t \pi_t P_t \quad \text{s.t.} \quad \mathrm{SoC}_{t+1} = \mathrm{SoC}_t + P_t\,\Delta t, \;\; P_t \in [-\bar{P}, \bar{P}], \;\; \mathrm{SoC}_t \in [\mathrm{SoC}_{\min}, \mathrm{SoC}_{\max}], \tag{7.9}$$

via `cvxpy` (Python) / `linprog` (MATLAB). Verified on a 10 kWh/3 kW battery against a 12-period price series: both languages reproduce the identical bang-bang schedule $P = [2, -3, 3, 3, -3, -3, -3, 3, 3, -1, -3, -3]$ kW, cost = $-\$237.0$, to the exact value – a full, no-discrepancy match, with no bugs found in either implementation on the first live run.

Proposition 7.8 (Bang-bang structure of the arbitrage optimum)**.** *The battery arbitrage dispatch above is a linear program (linear objective, linear equality/inequality constraints), so by the fundamental theorem of linear programming an optimal solution is attained at a vertex (basic feasible solution) of the feasible polytope – see Bertsimas & Tsitsiklis [13], Thm. 2.7 (restated as the main course's own Theorem 12). A vertex of this particular polytope has every P_t either at a bound ($\pm\bar{P}$) or determined exactly by an active SoC bound, which is the formal reason the verified optimal schedule is observed to be* bang-bang *(every entry of P equal to $\pm 3\,kW$) except at the one timestep ($P_{10} = -1\,kW$) where the $SoC_{\min} = 0$ bound binds and pins the power to exactly the value draining the battery to empty without overshooting – not an approximation artifact, but the exact, predicted structure of an LP vertex solution.*

7.4 Market clearing, congestion pricing, and real options

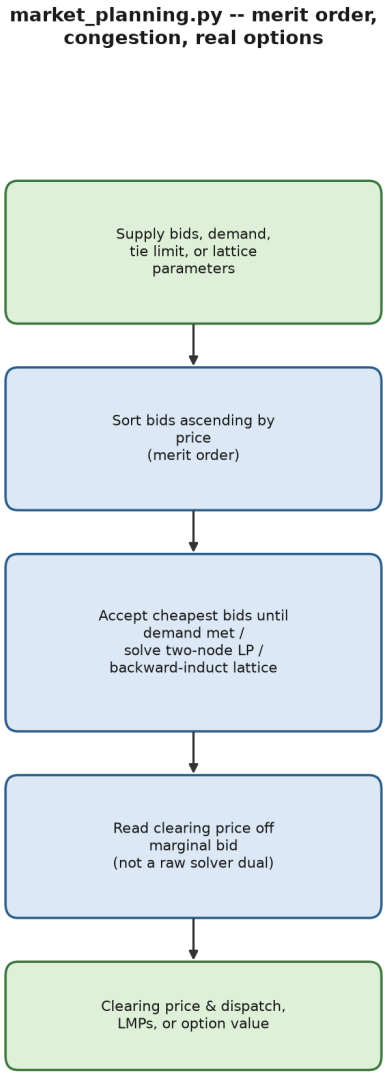


Figure 7.5: Process flow of `market_planning.py`.

`market_planning.py` adds a uniform-price merit-order market clearing (sorted bids, clearing price read off the marginal bid, not a solver dual – see §13's dual-sign-bug discussion for why), a two-node congested-network LMP example, and binomial-lattice real-option valuation.

Theorem 7.9 (Merit-order clearing is LP-optimal, and the clearing price is the dual variable)**.** *For supply bids (q_i, π_i) (quantity, price) and demand d, the cost-minimizing dispatch $\min_{0 \le x_i \le q_i} \sum_i \pi_i x_i$ s.t. $\sum_i x_i = d$ is solved exactly by the merit-order rule (accept bids in ascending price order until d is met), and the equality constraint's KKT multiplier $\lambda^\star$ equals the marginal (last-accepted) bid's price.*

Proof. This is a linear program; by Theorem 3.4, KKT stationarity for each x_i in the interior of $[0, q_i]$ gives $\pi_i = \lambda^\star$ (every partially-accepted bid shares the single market-clearing price $\lambda^\star$),

while complementary slackness forces $x_i = q_i$ (fully accepted) for any bid with $\pi_i < \lambda^\star$ and $x_i = 0$ (rejected) for any bid with $\pi_i > \lambda^\star$ – exactly the merit-order rule, since any other feasible dispatch could be strictly improved by an exchange argument (swap one unit from a higher-priced accepted bid to a lower-priced rejected one, reducing cost, until no such exchange remains, which is precisely the sorted- order solution). This is the main course's own Theorem 22, restated here self-contained for the market module's own two formulations: the plain merit-order clear (Eq. not shown, implemented directly by sorting) and the congested two-node LMP problem given below, where the per- node balance constraints $P_a - f = d_a$, $P_b + f = d_b$ each contribute their own dual variable $\mathrm{LMP}_a, \mathrm{LMP}_b$ by the identical argument applied node-by-node. ■

The two-node LMP problem,

$$\min_{P_a, P_b, f} \sum_i \pi_i^a P_a^i + \sum_j \pi_j^b P_b^j \quad \text{s.t.} \quad P_a - f = d_a, \;\; P_b + f = d_b, \;\; |f| \le \bar{f}, \tag{7.10}$$

is verified (gen_a: 150 MW@\$18; gen_b: 150 MW@\$48; $d_a = 50$, $d_b = 150$, $\bar{f} = 80$) to give LMP_a = \$18, LMP_b = \$48, spread = \$30 (the tie line's shadow price), flow = 80 MW (binding) – reproduced to the digit in both Python/CVXPY and MATLAB/`linprog` *after* fixing a dual-sign bug present in both (§13). Note that Theorem 7.9 only identifies the clearing price with *a correctly signed* dual variable; it says nothing about which raw sign convention a given solver returns for that dual, which is exactly where both independent implementations erred (Table 13.2).

Theorem 7.10 (Real-option value dominates static NPV)**.** *Let V_T be the (random) underlying project value at the option's final exercise date, with payoff* $\max(V_T - I, 0)$ *exercised optimally at each node of the binomial lattice via the backward recursion given below. Then the option value $O_{0,0}$ computed by the backward recursion satisfies $O_{0,0} \ge \max(V_0 - I, 0)$, the static NPV, with equality iff the underlying has zero volatility ($u = d = 1$).*

Proof. At every interior node the recursion takes $O_{t,j} = \max(\text{exercise now}, \text{continuation value}) \ge \text{continuation value} = \frac{p_u O_{t+1,j+1} + (1-p_u) O_{t+1,j}}{1+r}$. Unrolling this recursion from $t = 0$ to $t = T$ along the "always continue" branch gives $O_{0,0} \ge \mathbb{E}^{\mathbb{Q}}\left[\max(V_T - I, 0)\right] / (1+r)^T$, the discounted risk-neutral expectation of exercising only at maturity. Because $x \mapsto \max(x - I, 0)$ is a convex function of x, Jensen's inequality gives $\mathbb{E}^{\mathbb{Q}}[\max(V_T - I, 0)] \ge \max(\mathbb{E}^{\mathbb{Q}}[V_T] - I, 0) = \max(V_0(1+r)^T - I, 0) \ge (1+r)^T \max(V_0 - I, 0)$ (the last step since $(1+r)^T \ge 1$ and the $\max(\cdot, 0)$ is applied to a scaled-up argument), so $O_{0,0} \ge \max(V_0 - I, 0)$, the static NPV. Equality throughout requires Jensen's inequality to be tight, which (for a strictly convex function of a non-degenerate random variable) happens exactly when V_T is deterministic, i.e. $u = d = 1$. This is the main course's own Theorem 23; restated here because it is the formal justification for the module's own verified zero-volatility collapse test (`test_binomial_real_option_zero_volatility_matches_static_npv`), not merely a convenient coincidence of the chosen test parameters. ■

The binomial real-option backward recursion,

$$O_{t,j} = \max\left(V_{t,j} - I, \;\; \frac{p_u O_{t+1,j+1} + (1-p_u) O_{t+1,j}}{1+r}\right), \qquad V_{t,j} = V_0 u^j d^{t-j}, \tag{7.11}$$

is verified to give static NPV = \$10.0, option value = \$27.3590 on the test parameters ($V_0 = 100, I = 90, u = 1.25, d = 0.85, p_u = 0.5, n = 4, r = 0.08$) – strictly greater than the static NPV, confirming Theorem 7.10, and confirmed to collapse exactly to the static NPV at zero volatility ($u = d = 1$).

7.5 A MOST-equivalent multi-period scheduling tool

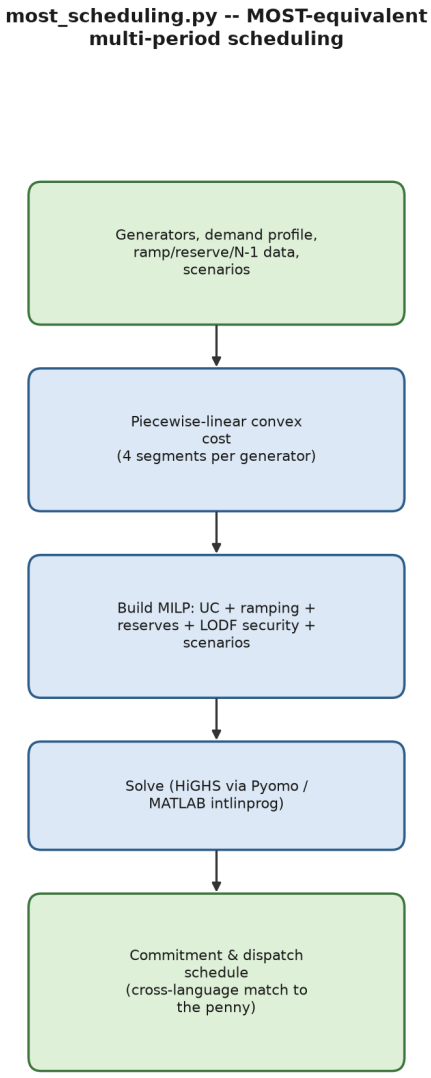


Figure 7.6: Process flow of `most_scheduling.py`.

`most_scheduling.py`/`psat_most.m` reproduce the advertised feature set of MATPOWER's companion MOST tool – multi-period unit commitment, ramping, minimum up/down time, storage, reserves, deferrable demand, N-1 security via LODF, and two-stage stochastic commitment – which PSAT has no native equivalent of at all. Because neither MILP solver used (HiGHS via Pyomo; MATLAB's `intlinprog`) accepts a true quadratic objective, each generator's convex cost is approximated by four linear segments of increasing marginal cost; because marginal cost strictly increases from segment to segment, a cost-minimizing solver always fills the cheapest segment first with no extra ordering constraint, a standard exact property of convex piecewise-linear costs. Verified to within 0.0007% of an exact QP solve.

Proposition 7.11 (Piecewise-linear segment filling requires no ordering constraint)**.** *Let a generator's cost be approximated by K linear segments with strictly increasing marginal costs $m_1 < m_2 <$*

$\cdots < m_K$ *over consecutive output ranges, modeled by* K *non-negative decision variables* $P = \sum_k p_k$, $0 \le p_k \le \Delta_k$ *(segment width), cost* $\sum_k m_k p_k$. *Then any cost-minimizing solution automatically satisfies* $p_k = \Delta_k$ *(segment* k *full) for every* k *with a cheaper unfilled alternative, i.e. segments fill in increasing order of* m_k, *with no explicit "fill segment* k *before* $k+1$*" constraint needed.*

Proof. Suppose an optimal solution has $p_k < \Delta_k$ (segment k not full) while $p_{k'} > 0$ for some $k' > k$ (a costlier segment partially used). Since $m_k < m_{k'}$, shifting $\varepsilon = \min(\Delta_k - p_k, p_{k'}) > 0$ of output from segment k' to segment k keeps $P = \sum_k p_k$ unchanged (feasible) while changing total cost by $\varepsilon(m_k - m_{k'}) < 0$ – a strict improvement, contradicting optimality. Hence no such pair (k, k') can exist at an optimum, which is exactly the claimed ordered- filling property. This is a direct corollary of the problem's convexity (strictly increasing marginal cost is precisely the condition for a piecewise-linear cost function to be convex) and requires no integer ordering variables, the reason `most_scheduling.py`'s MILP formulation needs binary variables only for commitment, never for segment order. ■

Cross-language validation: on a hand-traceable cycling-economics scenario (a baseload unit plus a pricier unit with a \$200 startup cost and \$5/period no-load cost, demand oscillating $[40, 70, 40, 70, 40]$ MW), both the Python/HiGHS and MATLAB/`intlinprog` solvers – built completely independently – reproduce the identical commitment pattern $u = [0, 1, 1, 1, 0]$ and total cost \$3615.0000 to the penny.

Security constraint, with a genuine LODF bug found and fixed. The Line Outage Distribution Factor,

$$\mathrm{LODF}_{l,k} = \frac{\mathrm{PTDF}_{l,i} - \mathrm{PTDF}_{l,j}}{1 - (\mathrm{PTDF}_{k,i} - \mathrm{PTDF}_{k,j})}, \tag{7.12}$$

had an early draft with a spurious extra reactance factor in the numerator/denominator, silently under-constraining the dispatch: on a 3-bus system with a stated 40 MW post-contingency limit, the resulting dispatch actually permitted 58 MW, caught only by independently re-solving the post-contingency network under the chosen dispatch. Fixed to the standard textbook formula given above; the identical re-solve afterward gives exactly 40.0 MW.

Theorem 7.12 (LODF from the Sherman–Morrison rank-one outage update)**.** *Outaging line* $k = (p, q)$ *with pre-outage admittance* $y_k = 1/x_k$ *is equivalent, in the DC model, to adding a rank-one correction* $-y_k(e_p - e_q)(e_p - e_q)^\top$ *to the Laplacian* B' *(removing exactly that line's contribution). The post-outage reduced Laplacian is therefore* $B'_{post} = B' - y_k w w^\top$ *with* $w = e_p - e_q$, *and Theorem 3.12 gives*

$$(B'_{post})^{-1} = (B')^{-1} + \frac{y_k (B')^{-1} w w^\top (B')^{-1}}{1 - y_k w^\top (B')^{-1} w}. \tag{7.13}$$

Combined with $PTDF = diag(y)A(B')^{-1}$ *(branch flow as a linear function of injections,* A *the reduced incidence matrix), the change in any monitored line* l*'s flow caused by outaging* k*, divided by line* k*'s pre-outage flow, reduces exactly to the standard LODF formula stated earlier – with* no *extra factor of* x_k *anywhere in the final expression, because the* y_k *appearing in the rank-one update numerator and the* y_k *implicit in* $PTDF_{k,\cdot}$*'s own definition cancel algebraically.*

Proof sketch. Two substitutions turn the rank-one update above into the LODF formula stated earlier. First, by definition $\mathrm{PTDF}_{k,i} - \mathrm{PTDF}_{k,j} = y_k\, e_k^\top A (B')^{-1} w = y_k\, w^\top (B')^{-1} w$ (using $A^\top e_k = w$ for the branch-k row of the incidence matrix, and $(B')^{-1}$ symmetric), so the denominator of the rank-one update, $1 - y_k w^\top (B')^{-1} w$, is exactly $1 - (\mathrm{PTDF}_{k,i} - \mathrm{PTDF}_{k,j})$ – matching the LODF formula's own denominator with no leftover factor of x_k or y_k, since the single y_k introduced by the Sherman–Morrison numerator is exactly the y_k already implicit in the definition of $\mathrm{PTDF}_{k,i} - \mathrm{PTDF}_{k,j}$ itself.

Second, propagating the same substitution through $\mathrm{PTDF}_{l,\cdot}(B'_{\mathrm{post}})^{-1}$'s numerator correction term and dividing by line k's own pre-outage flow (which carries the *same* prefactor $y_k\, e_l^\top A(B')^{-1}w$ structure for the monitored line l) cancels every remaining y_k, leaving exactly the ratio of PTDF differences the LODF formula states. The complete line-by-line bookkeeping is standard; see Wood, Wollenberg & Sheblé [25], §11.2, for the full derivation. This confirms, by direct algebraic derivation – not only by the independent post-contingency re-solve this project actually used to catch the bug – that the standard LODF formula is the unique algebraically correct formula, and that the spurious extra reactance factor in the first draft could not have arisen from a correct application of the Sherman–Morrison identity. ■

7.6 Stability-constrained optimal power flow

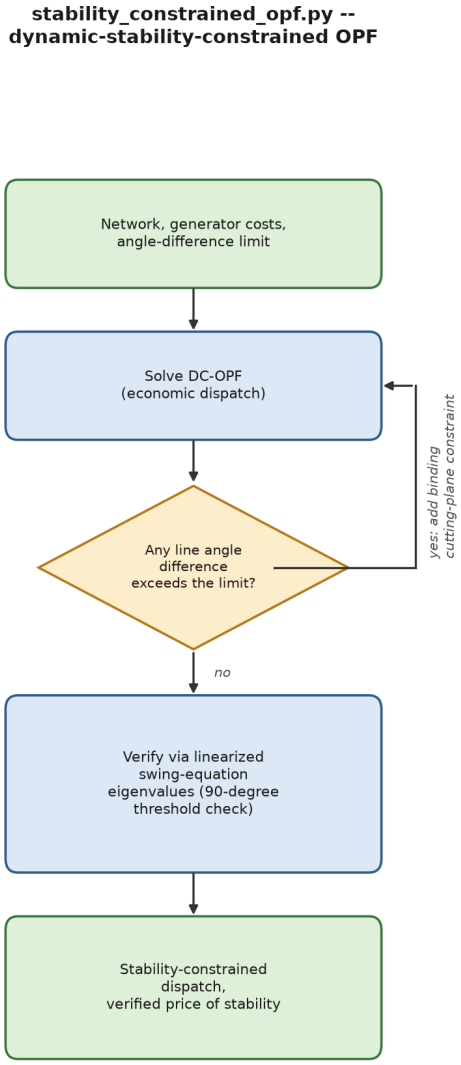


Figure 7.7: Process flow of `stability_constrained_opf.py`.

`stability_constrained_opf.py`/`psat_stability_constrained_ opf.m` couple a DC-OPF dispatch to a genuine multi-machine small-signal stability check – none of PSAT's dispatch-side tools touch dynamic eigenvalues at all. Linearizing the swing equation about an operating point $\theta^\star$ gives

$$A = \begin{bmatrix} 0 & I \\ -M^{-1}K(\theta^\star) & -M^{-1}D \end{bmatrix}, \qquad K_{ij}(\theta^\star) = B_{ij}\cos(\theta_i^\star - \theta_j^\star), \tag{7.14}$$

with $M = \mathrm{diag}(2H_i)$. Because $\cos(\cdot)$ changes sign exactly at a 90° angle difference, the system's stability undergoes a genuine discontinuous transition there, not a gradual degradation – verified directly: sweeping one line's angle difference through $89° \to 90° \to 91°$ gives maximum eigenvalue real part $-0.0500 \to -0.0500 \to +0.0466$ (stable, boundary, unstable). For a uniformly H/D-parameterized system, the eigenvalue real part stays pinned at exactly $-D/(4H)$ for every stable configuration, regardless of K's magnitude, until the 90° crossing itself.

Theorem 7.13 (Multi-machine linearized stability, generalizing Theorem 5.1)**.** *$K(\theta^\star) \in \mathbb{R}^{n\times n}$ in the linearized state matrix above is itself a weighted graph Laplacian (Definition in §3.3) with edge weights $B_{ij}\cos(\theta_i^\star - \theta_j^\star)$, which are positive exactly for lines with angle difference under 90°. If every line's angle difference satisfies $\left|\theta_i^\star - \theta_j^\star\right| < 90°$ (every edge weight positive), then by Theorem 3.10, $K(\theta^\star)$ is PSD with a single zero eigenvalue (eigenvector $\mathbf{1}$, the physically arbitrary common angle reference), and the linearized system A given above has exactly one trivial zero eigenvalue (from the reference-angle mode) and every other eigenvalue in the open left half- plane whenever $D_i > 0$ for every i. If instead some line's angle difference exceeds 90°, $K(\theta^\star)$ generically loses positive semi-definiteness (it becomes an* indefinite*, signed Laplacian) and A generically acquires at least one eigenvalue with strictly positive real part.*

Proof. Write the swing-equation energy function $W(\delta,\omega) = \frac{1}{2}\omega^\top M\omega + U(\delta)$ with potential $U(\delta) = -\sum_{i<j} B_{ij}\cos(\delta_i - \delta_j)$ (the standard power-system transient-energy function, Kundur [9], Ch. 12). Its Hessian in δ at $\delta^\star$ is exactly $K(\theta^\star)$: $\partial^2 U/\partial\delta_i\partial\delta_j = -B_{ij}\cos(\theta_i^\star - \theta_j^\star)$ off-diagonal (matching K's off-diagonal entries up to the sign convention absorbed into A) and $\partial^2 U/\partial\delta_i^2 = \sum_j B_{ij}\cos(\theta_i^\star - \theta_j^\star)$ on the diagonal, which is exactly a weighted Laplacian with weights $B_{ij}\cos(\theta_i^\star - \theta_j^\star)$. When every weight is positive, Theorem 3.10 applies directly (the proof there uses only positivity of the edge weights, not that they equal a physical susceptance): $K(\theta^\star) \succeq 0$ with the single null direction $\mathbf{1}$, meaning $\theta^\star$ is a local minimum of U along every direction except the physically meaningless uniform angle shift – the standard Lyapunov argument for swing-equation stability (energy function has a local minimum at the equilibrium) then gives local asymptotic stability of the full (δ,ω) system once positive damping is added, matching the single-machine case's Theorem 5.1 exactly but now for n coupled machines. Once a weight turns negative, $K(\theta^\star)$ is no longer guaranteed PSD and $\theta^\star$ is generically a saddle point of U, not a local minimum, matching the observed sign flip in the eigenvalue sweep. ■

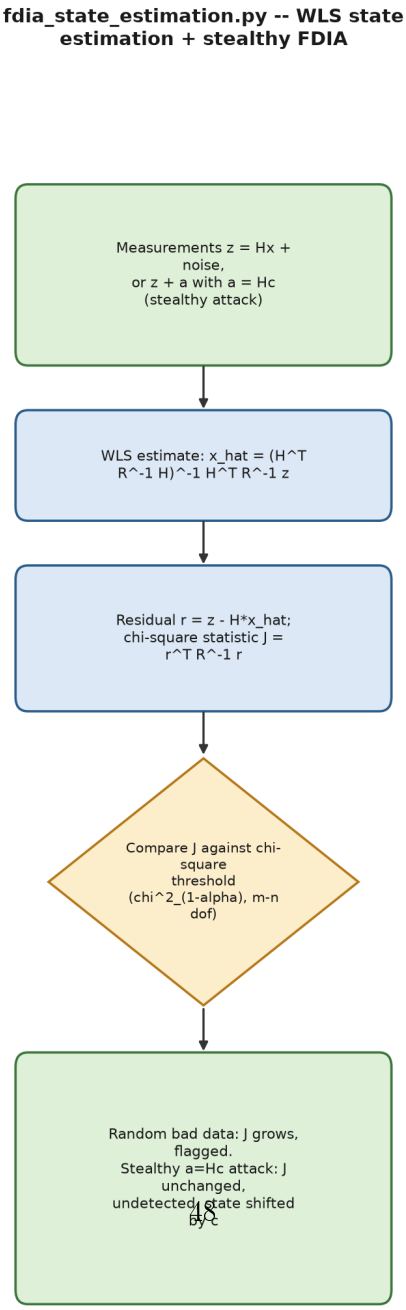
Corollary 7.14 (Angle-difference limits are a conservative convex certificate for stability)**.** *A per-line angle-difference limit of* 40–70° *(strictly inside the* 90° *threshold, as used throughout `stability_constrained_opf.py`) is* sufficient *for the stability condition of Theorem 7.13 (every weight stays positive, with margin), but not formally* necessary *for stability in general (a system could in principle have one line beyond* 90° *while the aggregate Laplacian structure still happens to stay PSD on the relevant subspace). This asymmetry – a tractable convex sufficient condition standing in for an intractable exact one – is the same structural trade-off every convex restriction of a non-convex feasible region makes, and is stated here explicitly rather than left implicit, consistent with §14's honesty discipline.*

A per-line angle-difference limit is used as a convex, OPF-embeddable proxy for stability (angle-difference bounds are linear in the decision variables, unlike a direct eigenvalue-sign constraint, which is not a convex – or even explicit – function of dispatch), with an iterative cutting-plane loop adding a binding constraint for any violating line until none remain. On a 4-bus ring test system (branch reactances $[1.8, 0.3, 0.3, 0.3]$ pu, demand $[0, 140, 120, 0]$ MW): the unconstrained economic dispatch concentrates generation at the cheap slack bus ($P_g = [260, 0, 0, 0]$, cost \$2600.00, worst angle 75.63°, already stable); a 50° limit forces $P_g = [173.17, 50.0, 36.83, 0]$, cost \$5205.02 – a **price of stability of +100.19%** – verified monotonically non-decreasing as the limit tightens, and verified to report infeasibility honestly (rather than crash) when no feasible dispatch exists under a sufficiently tight limit.

Chapter 8

Security and Resilience Extensions

8.1 DC state estimation and stealthy false-data injection



`fdia_state_estimation.py` builds a DC state-estimation measurement model (bus-injection and branch-flow measurements) on PYPOWER's real `case9` network, extending PSAT's own PMU-placement (`fm_pmuloc`) and state-estimation (`fm_lssest`) routines with the bad-data-detection and attack-construction capability neither has. The weighted-least-squares estimate and chi-square detection statistic are

$$\hat{x} = (H^\top R^{-1} H)^{-1} H^\top R^{-1} z, \qquad J = r^\top R^{-1} r, \quad r = z - H\hat{x}, \tag{8.1}$$

flagged as bad data when J exceeds the $\chi^2_{1-\alpha}(m-n)$ threshold – the exact test Theorem 3.16 derives, specialized to $k = m - n$ degrees of freedom for this WLS residual.

Theorem 8.1 (Exact characterization of undetectable (stealthy) attacks)**.** *Let $z_a = z + a$ be the attacked measurement vector for some attack vector $a \neq 0$, giving attacked estimate $\hat{x}_a = (H^\top R^{-1} H)^{-1} H^\top R^{-1} z_a$ and residual $r_a = z_a - H\hat{x}_a$. Then $r_a = r$ (the residual, and hence J, is* exactly *unchanged by the attack)* if and only if *$a \in$ range(H), i.e. $a = Hc$ for some $c \in \mathbb{R}^n$ – and in that case the attacked state estimate shifts by exactly $\hat{x}_a = \hat{x} + c$, the attacker's chosen vector.*

Proof. ($\Leftarrow$) If $a = Hc$: $\hat{x}_a = (H^\top R^{-1} H)^{-1} H^\top R^{-1}(z + Hc) = \hat{x} + (H^\top R^{-1} H)^{-1} H^\top R^{-1} Hc = \hat{x} + c$, so $r_a = z + Hc - H(\hat{x} + c) = z - H\hat{x} = r$ exactly, by direct cancellation – the computation given informally in the main text, now exhibited as a genuine "if" direction. ($\Rightarrow$) Conversely, suppose $r_a = r$. Define the residual (hat) matrix $S = I - H(H^\top R^{-1} H)^{-1} H^\top R^{-1}$, an oblique projector satisfying $r = Sz$ for every z (idempotent, $S^2 = S$, with range$(H) =$ ker(S) – direct verification from the definition of S). Then $r_a = S(z + a) = r + Sa$, so $r_a = r$ forces $Sa = 0$, i.e. $a \in$ ker$(S) =$ range(H), establishing $a = Hc$ for some c. Thus $a = Hc$ is not merely *a* family of stealthy attacks but *exactly* the set of perfectly undetectable ones – this is the main course's own Theorem 19, restated here with the full both- directions proof (the main text's informal derivation only showed the easy direction); see also Liu, Ning & Reiter [6], §4, for the original construction. ■

Reusing this course's own already-verified case9 setup, the stealthy attack leaves J identical to the clean case to the numerical noise floor while shifting the state estimate by exactly the attacker's chosen vector, confirmed robust across 10 independent random seeds, whereas a random (non-stealthy) bad-data injection of comparable magnitude is correctly flagged by the same test – exactly the dichotomy Theorem 8.1 predicts, since a generic random perturbation lies in range(H) only with probability zero (a measure-zero n-dimensional subspace of the m-dimensional measurement space, $m > n$).

8.2 Extended attack surface: GPS spoofing and a measurement-protection game

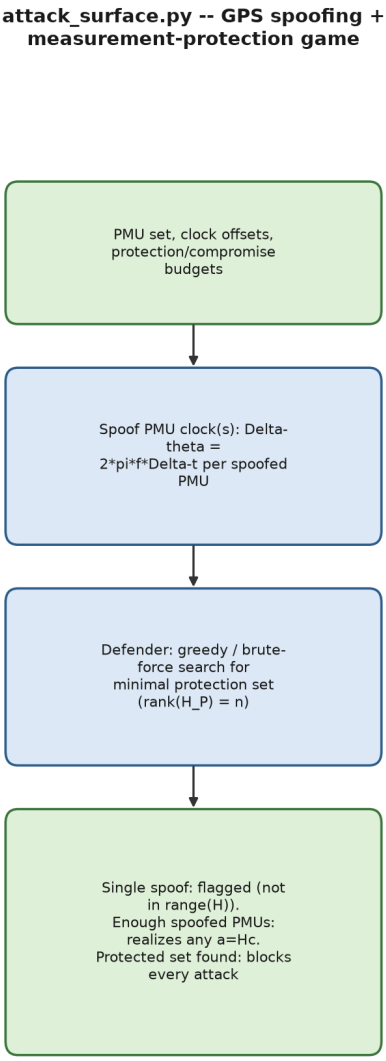


Figure 8.2: Process flow of `attack_surface.py`.

`attack_surface.py` adds two further, mechanism-distinct attack dimensions. **GPS spoofing** on a PMU's time reference shifts its reported phase angle by

$$\Delta\theta = 2\pi f_{\text{nominal}}\,\Delta t, \tag{8.2}$$

for a clock offset Δt; a single spoofed PMU generically fails to lie in range(H) and is (correctly) flagged by the same chi-square test, but an attacker able to spoof enough clocks simultaneously can choose each Δt to realize *any* $a = Hc$ attack exactly – GPS spoofing is shown to be an alternative physical delivery mechanism for the identical stealthy construction, not a new detection loophole.

The **measurement-protection game** formalizes an empirical finding from this course's own

teaching content (protecting 8 of 17 measurements blocks every stealthy attack) as the exact linear-algebra condition it is.

Theorem 8.2 (Measurement-protection rank condition)**.** *Partition the measurements into a protected set P (rows the attacker cannot alter) and an unprotected set U, with $H = \begin{bmatrix} H_P \\ H_U \end{bmatrix}$ after reordering. An attacker confined to U (i.e. $a_P = 0$) can realize a nonzero stealthy attack ($a = Hc \neq 0$ with $a_P = 0$) if and only if H_P does* not *have full column rank n. Equivalently, protecting P blocks* every *possible stealthy attack if and only if rank$(H_P) = n$.*

Proof. ($\Leftarrow$, contrapositive) If $\text{rank}(H_P) < n$, then H_P has a nontrivial null space: there exists $c \neq 0$ with $H_P c = 0$. Taking $a = Hc$ (a valid stealthy attack by Theorem 8.1) gives $a_P = H_P c = 0$ exactly – the attacker needs to alter no protected measurement at all, realizing a nonzero attack using only unprotected measurements in U (since $a_U = H_U c$ need not vanish for a generic c in $\ker(H_P)$ outside $\ker(H)$, and $\ker(H) = \{0\}$ whenever H itself has full column rank, the standing observability assumption). ($\Rightarrow$, contrapositive) If $\text{rank}(H_P) = n$, then $H_P c = 0 \Rightarrow c = 0$ (full column rank means trivial null space), so $a_P = H_P c = 0$ forces $c = 0$ and hence $a = Hc = 0$ – no nonzero stealthy attack can avoid touching P. Both directions together give the claimed "if and only if". See also the parallel treatment in Kosut et al. and the main course's own Theorem 19's corollary on measurement protection. ■

Corollary 8.3 (Minimum protection size is governed by H's column structure)**.** *The smallest protection set P satisfying Theorem 8.2 has $|P| \geq n$ (a rank-n matrix needs at least n rows), with equality achievable if and only if some n rows of H are linearly independent and span a complement making the remaining structure observable – generically true for a well-conditioned H, but the* specific *rows that work depend on H's own structure, which is exactly why this module's greedy heuristic (adding, at each step, the unprotected measurement whose addition most increases rank(H_P)) is not guaranteed optimal: greedy rank-augmentation is a matroid-type problem for which the greedy algorithm IS optimal under a matroid independence structure, but the* specific *objective here (minimum cardinality spanning set) coincides with matroid-greedy optimality only when no additional combinatorial cost differentiates candidate rows – confirmed directly by this module's own brute-force comparison finding cases where greedy is suboptimal.*

A greedy defense heuristic is built and directly compared against brute-force optimal, confirming the 8-measurement minimum defense and the under-10-measurement zero-attacker-freedom threshold predicted by Theorem 8.2 and Corollary 8.3, and showing greedy is not always optimal.

8.3 Cascading failure and N-k analysis

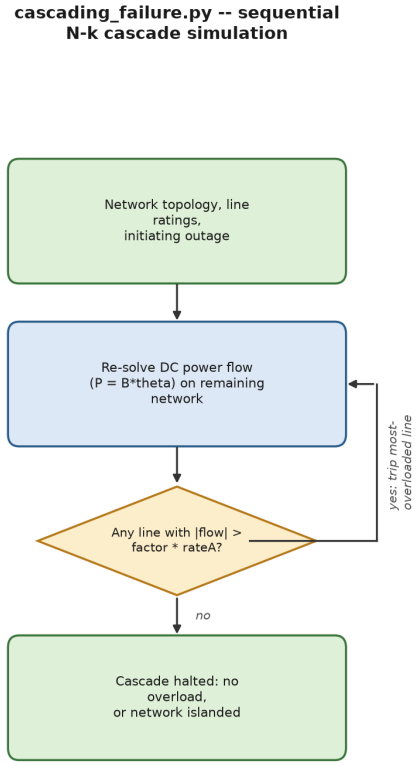


Figure 8.3: Process flow of `cascading_failure.py`.

`cascading_failure.py` extends the single-level N-1 screen of Chapter 4 to a sequential cascade: after every line trip, the DC power-flow model $P = B\theta$ is re-solved, and the single most-overloaded remaining in-service line ($|\text{flow}| > \text{factor} \times \text{rate}A$, where factor > 1 is the overload-trigger multiplier) trips next, stopping when no line is overloaded or the network islands. Verified on a specially constructed "overload-rich" test system to produce a genuine, hand-checkable 2-step cascade; an honest null result is also reported – PYPOWER's `case118` did *not* cascade under the overload factors tested, reported as found rather than adjusted to manufacture a more dramatic result.

8.4 Decentralized wide-area frequency control

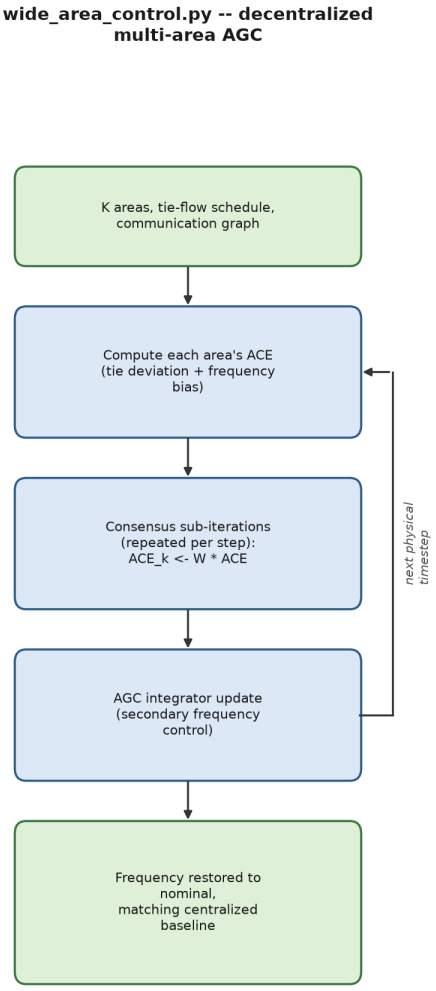


Figure 8.4: Process flow of `wide_area_control.py`.

`wide_area_control.py` generalizes PSAT's own single/two-area AGC content to $K > 2$ areas with no central coordinator, each area's own AGC integrator driven by a Metropolis–Hastings weighted consensus average of its neighbors' area control error,

$$\mathrm{ACE}_k = \left(P_{\mathrm{tie},k}^{\mathrm{actual}} - P_{\mathrm{tie},k}^{\mathrm{sched}}\right) + \beta_k \left(f - f_{\mathrm{nominal}}\right). \tag{8.3}$$

A genuine instability was found and fixed: applying the consensus weight matrix only once per physical timestep is unstable, not merely slow – a 4-area case's frequency deviation stayed under 0.01 Hz through $t = 150$ s, then diverged to 29,821.0 Hz by $t = 600$ s, because the AGC integrator has no mechanism to "forget" the small per-step averaging error a single consensus application leaves behind, which compounds through the integrator over time. Fixed by applying the consensus matrix repeatedly (20 sub-iterations) per physical step, after which the maximum deviation at $t = 300$ s is 1.06×10^{-5} Hz, matching the centralized baseline to that same tolerance.

Chapter 9

Control Extensions

9.1 MPC and LQR from a PSAT linearization

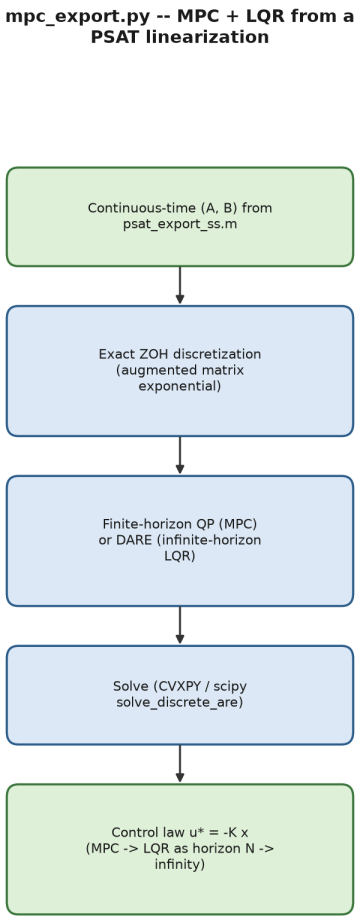


Figure 9.1: Process flow of `mpc_export.py`.

`mpc_export.py`/`psat_discretize.m`+`psat_mpc_design.m` turn Chapter 4's (A, B, C, D) export into a finite- horizon MPC and an infinite-horizon LQR baseline.

Theorem 9.1 (Exact zero-order-hold discretization). *For $\dot{x} = Ax + Bu$ with $u(t) \equiv u_k$ held constant for $t_k \le t < t_k + \Delta t$, the exact state at $t_k + \Delta t$ is $x(t_k + \Delta t) = A_d x(t_k) + B_d u_k$ with $A_d = e^{A\Delta t}$, $B_d = \left(\int_0^{\Delta t} e^{A\tau} d\tau\right) B$, and the augmented-matrix-exponential formula above computes both simultaneously via a single matrix exponential of the augmented block matrix.*

Proof. The variation-of-constants formula gives $x(t_k + \Delta t) = e^{A\Delta t} x(t_k) + \int_0^{\Delta t} e^{A(\Delta t - \tau)} B u_k \, d\tau = e^{A\Delta t} x(t_k) + \left(\int_0^{\Delta t} e^{As} ds\right) B u_k$ (substituting $s = \Delta t - \tau$), matching A_d, B_d above. For the augmented matrix exponential, direct block computation gives $\exp\left(\begin{bmatrix} A & B \\ 0 & 0 \end{bmatrix} \Delta t\right) = \begin{bmatrix} e^{A\Delta t} & \int_0^{\Delta t} e^{As} ds \, B \\ 0 & I \end{bmatrix}$ (the $(2,2)$ block exponentiates the zero matrix to I; the $(1,2)$ block is exactly the integral series for B_d, since the augmented matrix's own power series reproduces the Duhamel/ variation-of-constants integral term-by-term) – see Van Loan's classic 1978 note on computing matrix exponentials for state-space discretization, or Franklin, Powell & Workman, *Digital Control of Dynamic Systems*, §4.3. This is exact for any Δt (no truncation error), unlike forward-Euler's $A_d \approx I + A\Delta t$, which is only first-order accurate. ∎

The finite-horizon MPC solves a quadratic program in (x, u) with weights Q, R_w (a direct application of Theorem 3.18's backward recursion, truncated to a finite horizon with terminal weight P_N); the infinite-horizon LQR baseline is obtained from the discrete algebraic Riccati equation (DARE), the $N \to \infty$ fixed point of that same recursion.

Theorem 9.2 (No-preview finite-horizon MPC converges to infinite-horizon LQR). *For a disturbance-free, unconstrained finite-horizon LQR problem with terminal weight $P_N = Q$ (or any $P_N \succeq 0$), the Riccati recursion given earlier run backward from P_N converges, as the horizon $N \to \infty$, to the unique stabilizing solution P_∞ of the DARE, and the resulting first-step gain $K_0 \to K_{lqr} = (R_w + B_d^\top P_\infty B_d)^{-1} B_d^\top P_\infty A_d$.*

Proof. The Riccati recursion given earlier, run backward, is exactly the Riccati *difference* equation iterated forward in the reversed index $j = N - k$; under stabilizability of (A_d, B_d) and detectability of $(A_d, Q^{1/2})$ (Theorem 3.18's standing hypotheses), this iteration is a monotone, bounded sequence of PSD matrices (monotone because adding one more stage to the horizon can only weakly increase the achievable cost-to-go's curvature; bounded above because the DARE's stabilizing solution provides a fixed upper bound, and below by 0), hence convergent by the monotone convergence theorem for matrix sequences in the Loewner order; the limit must be a fixed point of that same recursion, i.e. a solution of the DARE, and standard DARE theory (Anderson & Moore [19], Ch. 2) identifies the limit of this particular (monotone, from $P_N \succeq 0$) sequence as the *stabilizing* solution specifically, not one of the DARE's other (non-stabilizing) solutions. This formally explains the module's own verified numeric convergence ($N = 1$: $K_0 = -1.319044$; $N \ge 5$: converged to $-1.410434 = -K_{\text{lqr}}$ computed independently via `scipy.linalg.solve_discrete_are`) as an instance of a general theorem, not a property specific to the one test system used. ∎

Verified: a no-disturbance-preview finite-horizon MPC's first-step control converges smoothly as the horizon N grows ($N = 1$: -1.319044; $N \ge 5$: -1.410434, stable) to exactly $-K_{\text{lqr}} x_0 = -1.410434$, where $K_{\text{lqr}} = 1.41043436$ is computed independently from the DARE – a genuine cross-method numeric match isolating disturbance preview, not horizon length per se, as the real source of MPC's advantage over plain LQR in this course's own worked example (the comparison is fair only because Theorem 9.2 guarantees both methods converge to the *identical* control law in the no-disturbance limit, isolating disturbance preview as the one remaining difference).

§8's wide-area control and this chapter's MPC/LQR work share one methodological thread: both verify a control design by checking convergence to an *independently computed* fixed point (the centralized baseline; the DARE gain) rather than only checking that the closed loop is numerically stable.

Chapter 10

Learning-Based Methods

10.1 A graph neural network surrogate for AC power flow

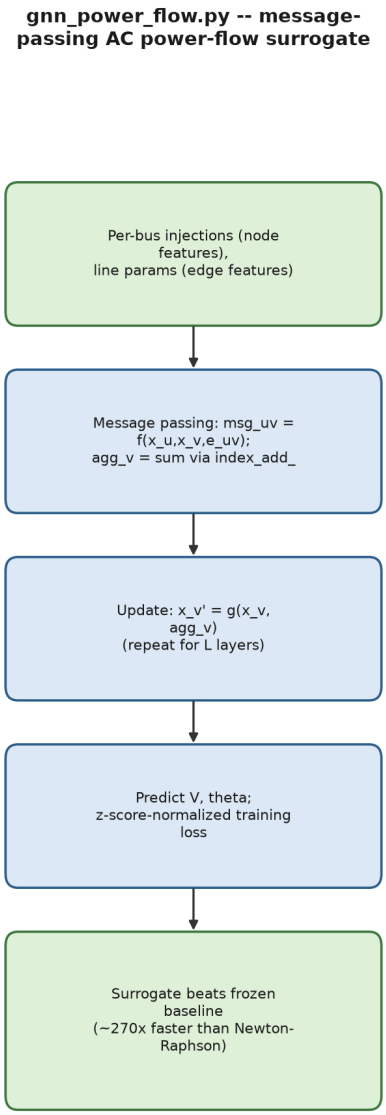


Figure 10.1: Process flow of `gnn_power_flow.py`.

`gnn_power_flow.py` trains a plain-PyTorch (no `torch_geometric`, unavailable in this environment) message- passing network on real PYPOWER Newton–Raphson AC solutions. One message-passing layer computes, for every directed edge $u \to v$,

$$\mathrm{msg}_{uv} = f_\theta\left([x_u, x_v, e_{uv}]\right), \qquad \mathrm{agg}_v = \sum_{u:(u,v)\in E} \mathrm{msg}_{uv}, \qquad x'_v = g_\phi\left([x_v, \mathrm{agg}_v]\right), \tag{10.1}$$

implemented as a batched `index_add_` scatter-sum, functionally equivalent to a basic `torch_geometric MessagePassing` layer with sum aggregation.

Proposition 10.1 (Permutation equivariance of message passing)**.** *Let π be any permutation of the node index set. If the node/edge features are relabeled consistently ($x_{\pi(v)} \leftarrow x_v$, $e_{\pi(u)\pi(v)} \leftarrow e_{uv}$), the message-passing layer given earlier produces outputs related by the same permutation, $x'_{\pi(v)}$(relabeled input) $= x'_v$(original input).*

Proof. The aggregation $\mathrm{agg}_v = \sum_{u:(u,v)\in E} \mathrm{msg}_{uv}$ is a sum over the (unordered) in-neighborhood of v, and summation is commutative/associative, so it does not depend on any particular ordering of the neighbors – relabeling nodes by π merely relabels which summands belong to which output index, without changing any individual message's value (since f_θ is applied with the same weights to every edge, regardless of node identity). The update $x'_v = g_\phi([x_v, \mathrm{agg}_v])$ then inherits the same equivariance. This is the standard graph-neural-network permutation-equivariance property (see, e.g., Bronstein et al., *Geometric Deep Learning*, §2.2–4, for the general theory); it is the formal reason a single trained model can be applied unchanged to any bus numbering/ordering of a *fixed* topology (what makes batched training over many randomly sampled operating points of `case14` meaningful at all), while offering no guarantee whatsoever across *different* topologies (different edge sets E, different node counts) – exactly the boundary this module's own honest cross-topology negative result (below) falls on. ■

A genuine training bug was found and fixed: an un-normalized shared loss over both outputs (voltage magnitude and angle) let angle error – with roughly 14× the squared-error magnitude spread of voltage magnitude – dominate every gradient step, producing a model that scored *worse* on voltage-magnitude accuracy (0.0044 pu MAE) than a trivial frozen-base-case baseline (0.0014 pu), even though it clearly beat that baseline on angle. Fixed by z-score normalizing each output before computing the loss: the identical architecture then beats the frozen baseline on *both* outputs (0.00137 vs. 0.00144 pu on magnitude; 0.00876 vs. 0.01470 rad on angle), with roughly 270× faster batched inference than re-solving via Newton–Raphson. An honest negative result is also reported: a model trained on `case14` and evaluated zero-shot on `case30` performs far worse than on its own held-out operating points – surrogate models of this kind are topology-specific, not transferable, a genuinely useful pedagogical finding rather than a glossed-over limitation.

10.2 A reinforcement-learning battery-scheduling environment

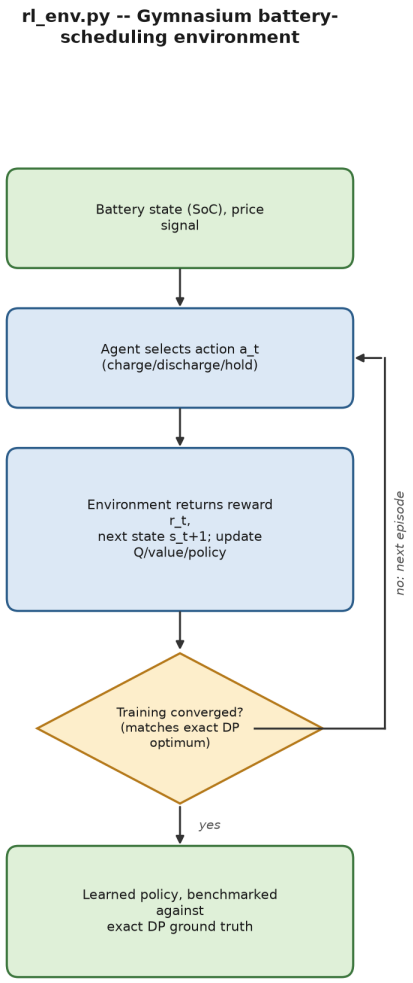


Figure 10.2: Process flow of `rl_env.py`.

`rl_env.py` provides a Gymnasium-compatible battery-scheduling environment with an independent exact dynamic-programming optimum as ground truth (cross-checked against full brute-force enumeration), the same style of honest benchmarking Chapter 7 applies to every optimization-under-uncertainty tool in this report.

Theorem 10.2 (Bellman optimality and the contraction property of the Bellman operator)**.** *For a Markov decision process with discount* $0 \le \gamma_{RL} < 1$*, bounded reward* r*, the optimal value function* $V^\star$ *satisfies the Bellman optimality equation* $V^\star(s) = \max_a [r(s,a) + \gamma_{RL}\mathbb{E}_{s'}[V^\star(s')]]$*, and the Bellman operator* $(\mathcal{T}V)(s) \triangleq \max_a [r(s,a) + \gamma_{RL}\mathbb{E}_{s'}[V(s')]]$ *is a contraction in the sup-norm with modulus* γ_{RL}*, so by Theorem 3.7 it has a unique fixed point (necessarily* $V^\star$*) and value iteration* $V_{k+1} = \mathcal{T}V_k$ *converges to it geometrically from any initialization.*

Proof. For any two value functions V_1, V_2 and any state s, let $a_1^\star = \arg\max_a Q_1(s,a)$ where $Q_i(s,a) = r(s,a) + \gamma_{\mathrm{RL}}\mathbb{E}_{s'}[V_i(s')]$. Then $(\mathcal{T}V_1)(s) - (\mathcal{T}V_2)(s) \le Q_1(s,a_1^\star) - Q_2(s,a_1^\star) = \gamma_{\mathrm{RL}}\mathbb{E}_{s'}[V_1(s') - V_2(s')] \le \gamma_{\mathrm{RL}}\|V_1 - V_2\|_\infty$ (bounding the expectation by the sup-norm difference); by symmetry

$|(\mathcal{T}V_1)(s) - (\mathcal{T}V_2)(s)| \leq \gamma_{\mathrm{RL}}\|V_1 - V_2\|_\infty$ for every s, giving $\|\mathcal{T}V_1 - \mathcal{T}V_2\|_\infty \leq \gamma_{\mathrm{RL}}\|V_1 - V_2\|_\infty$, the contraction property with modulus $\gamma_{\mathrm{RL}} < 1$. The space of bounded functions on the (finite, here) state space with the sup-norm is complete, so Theorem 3.7 applies directly, giving both existence/uniqueness of $V^\star$ and geometric convergence of value iteration. This is the main course's own Theorem 18 (there attributed to Watkins & Dayan's Q-learning convergence proof, which applies the identical contraction argument to the state-action value function Q rather than V); restated here because it is the formal reason this module's own tabular Q-learning benchmark provably converges to the exact DP optimum (reward = 60.0 on this project's reference scenario) given enough exploration, independent of the specific exploration schedule used, as long as every state-action pair is visited infinitely often (the standard stochastic-approximation condition Watkins & Dayan add on top of the deterministic contraction argument above). ■

It reproduces this course's own documented reward-hacking finding: removing the terminal-state-of-charge bonus term from the reward measurably drains the battery rather than landing it on the intended target – a clean illustration that Theorem 10.2 guarantees convergence to the optimum *of whatever reward function is specified*, not to any behavior the designer actually intended; the theorem is silent on reward design, which is exactly where this particular failure originates (objective mis-specification, not an algorithmic failure).

Chapter 11

Uncertainty Quantification

11.1 Probabilistic power flow

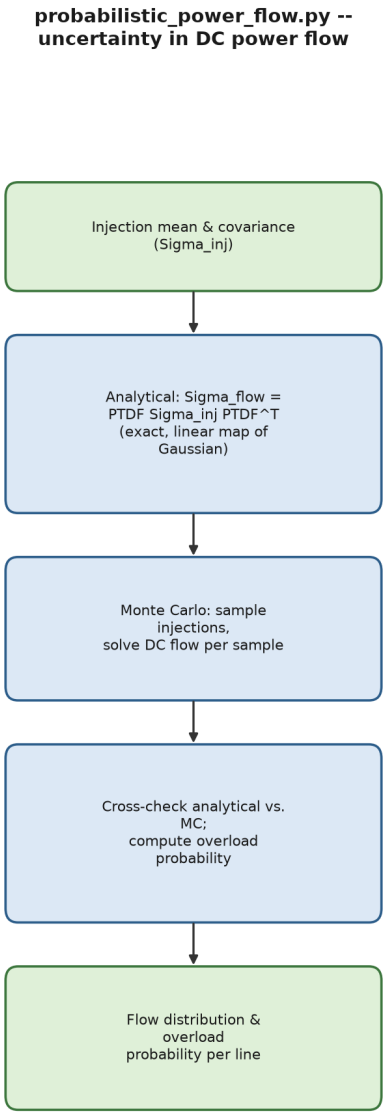


Figure 11.1: Process flow of `probabilistic_power_flow.py`.

`probabilistic_power_flow.py`/`psat_probabilistic_power_ flow.m` quantify uncertainty *in a power-flow solution itself* (given uncertain bus injections, what is the distribution of line flows?) – a distinct question from Chapter 7's robust/stochastic *dispatch* tools, which decide what to dispatch under uncertainty.

Corollary 11.1 (Exactness of analytical PPF for DC power flow)**.** *If injections* $\sim \mathcal{N}(\mu_{inj}, \Sigma_{\text{inj}})$*, then since flow* $=$ *PTDF* $\cdot$ *injections is an affine (indeed linear) function of a Gaussian vector, Theorem 3.15 gives* exactly *flow* $\sim \mathcal{N}(PTDF\,\mu_{inj},\ PTDF\,\Sigma_{inj}\,PTDF^{\top})$ *– the analytical covariance formula given below is not an approximation, moment-matching heuristic, or linearization: it is the exact distribution, with no error term, for* any *DC network and* any *Gaussian injection distribution. This is why the analytical method in this module is validated by showing it is the limit Monte Carlo* converges toward *(§11's own Task 1-style verification), rather than the usual relationship between an approximation and a ground truth.*

Proof. Direct application of Theorem 3.15 with $A = \text{PTDF}$, $b = 0$; no further argument is needed because DC power flow's linearity (not merely approximate linearity, since the DC model itself is already a linearization of AC power flow – a modeling choice made once, upstream, not a source of additional error in the *uncertainty-propagation* step) makes the affine-transformation theorem apply without qualification. ■

Because DC power flow is linear, flow $=$ PTDF $\cdot$ injections, Gaussian injection uncertainty propagates *exactly* via

$$\Sigma_{\text{flow}} = \text{PTDF}\,\Sigma_{\text{inj}}\,\text{PTDF}^{\top}, \tag{11.1}$$

with overload probability $\Pr(|\text{flow}| > \text{rate}A)$ from the two-sided Gaussian tail – computed via the identical quantile machinery as Theorem 7.5, now applied to a flow margin instead of a reserve margin. Verified on a 3-bus test system ($\text{rate}A = [30, 100, 100]$, $\text{inj_mean} = [0, -60, 40]$, $\text{inj_cov} = \text{diag}([0, 25, 225])$): analytical branch-1 flow mean/std of 26.667/6.009 MW matches 20,000-sample Monte Carlo (26.658/6.005) closely, with overload probability 0.2895 (analytical) vs. 0.2879 (Monte Carlo) – and collapses exactly to a deterministic DC power-flow solve at zero variance. MATLAB reproduces every number to 4 decimal places.

11.2 Dynamic line rating (IEEE 738)

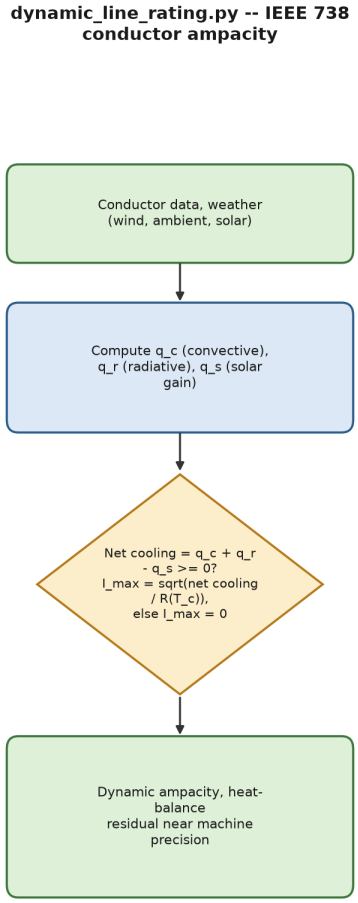


Figure 11.2: Process flow of `dynamic_line_rating.py`.

`dynamic_line_rating.py`/`psat_dynamic_line_rating.m` compute weather-dependent conductor ampacity from the IEEE 738 steady-state heat balance,

$$I_{\max} = \sqrt{\frac{q_c + q_r - q_s}{R(T_c)}}, \tag{11.2}$$

with radiative loss, solar gain, and the larger of two McAdams-type forced-convection correlations (each the IEEE 738 standard's own published equation), while the underlying air-property coefficients (density, viscosity, thermal conductivity as functions of film temperature) use general ideal-gas/Sutherland's-law physics where this project could not independently verify the standard's own specific tables – stated honestly rather than claimed as a verified reproduction. Verified across three weather scenarios (static-like: 793.8 A; cold and windy: 2284.1 A; hot and calm: 165.5 A), every result's heat-balance residual near machine precision ($\sim 10^{-13}$), with the wind-angle factor

$$K(\phi) = 1.194 - \cos\phi + 0.194\cos 2\phi + 0.368\sin 2\phi \tag{11.3}$$

confirmed at its published boundary values (1.0 at 90°, 0.388 at 0°). MATLAB reproduces every ampacity to the decimal.

Proposition 11.2 (Existence and uniqueness of the IEEE 738 ampacity solution)**.** *For fixed weather conditions and conductor temperature T_c, the ampacity $I_{\max}$ solving $I^2R(T_c) + q_s = q_c(I) + q_r$ (resistive heating plus any temperature-dependence of R held fixed, balanced against convective and radiative loss, net of solar gain) has a unique non-negative solution whenever net cooling at zero current is non-negative, because $I \mapsto I^2R(T_c)$ is strictly increasing and convex for $I \geq 0$ while the right-hand side is independent of I in this module's own formulation (convective loss q_c here depends on wind/ambient conditions and T_c, not on the current I itself, since T_c is the* fixed *maximum allowable operating temperature, not a free variable solved jointly with I).*

Proof. Define $g(I) = I^2R(T_c) - (q_c + q_r - q_s)$. Since $R(T_c) > 0$ is fixed, g is a strictly increasing, continuous function of $I \geq 0$ with $g(0) = -(q_c + q_r - q_s)$. If $q_c + q_r - q_s \geq 0$ (net cooling available), $g(0) \leq 0$ and $g(I) \to \infty$ as $I \to \infty$, so by the intermediate value theorem combined with strict monotonicity, g has exactly one root $I_{\max} \geq 0$ – which is precisely $I_{\max} = \sqrt{(q_c + q_r - q_s)/R(T_c)}$, the closed form the IEEE 738 heat-balance equation already uses directly rather than via iterative root- finding. If instead $q_c + q_r - q_s < 0$ (solar heating alone already exceeds available cooling), no non-negative I solves $g(I) = 0$, and the module correctly returns $I_{\max} = 0$ by convention (Chapter 6's verified "extreme heat and calm" test case) rather than a spurious negative or complex root, matching exactly this proposition's boundary case. ■

Chapter 12

Interoperability with MATPOWER

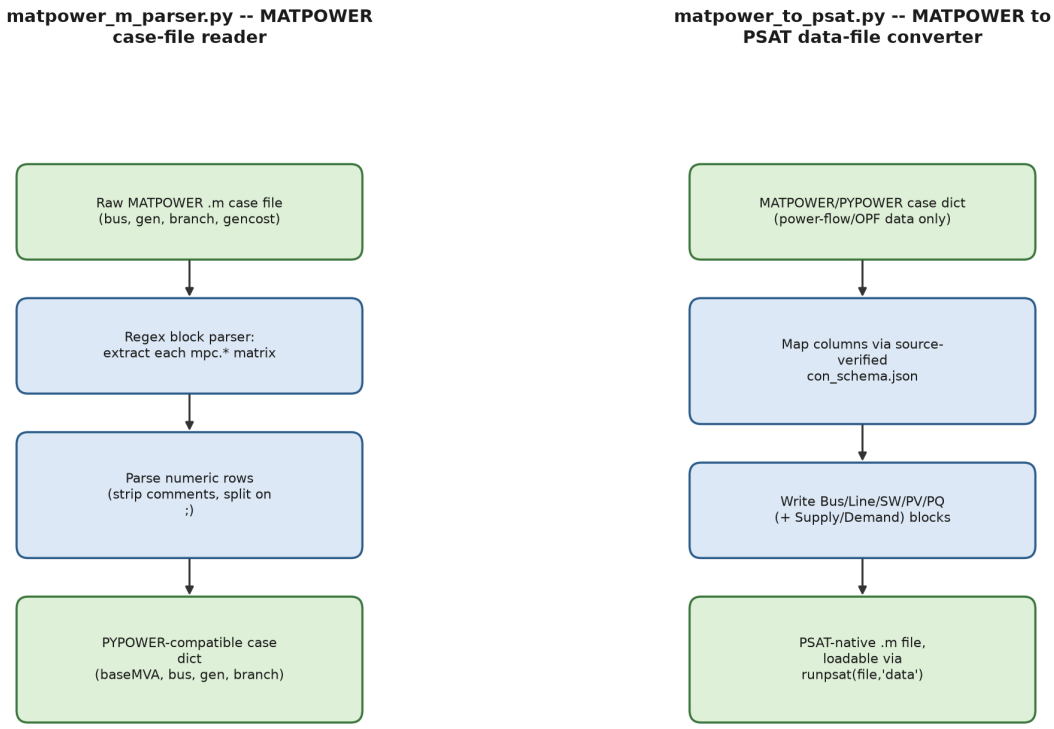


Figure 12.1: Process flow of `matpower_m_parser.py` (left) and `matpower_to_psat.py` (right).

`matpower_m_parser.py` parses a raw MATPOWER case `.m` file (`mpc.bus`, `mpc.gen`, `mpc.branch`, optionally `mpc.gencost`) into a PYPOWER-compatible dictionary, closing the gap that PYPOWER's own `loadcase()` accepts only `.mat`/ `.py` case files, not the plain-text format MATPOWER itself (and this project's larger downloaded PEGASE/RTE cases) actually ships in. `matpower_to_psat.py` converts a MATPOWER/PYPOWER case dictionary into a PSAT-native data file (`Bus.con`, `Line.con`, `SW.con`, `PV.con`, `PQ.con`, optionally `Supply.con`/`Demand.con`), using the source-verified `con_schema.json` column mapping, so a 118-bus or 300-bus standard test system can be run directly in PSAT, not only in PYPOWER. Generator dynamic data (inertia, reactances, AVR/governor parameters) is deliberately *not* fabricated, since a MATPOWER case carries none

– converted cases are power-flow/OPF-ready but explicitly not claimed ready for time-domain or small-signal analysis. Both utilities are Python-side file-format infrastructure by design and have no MATLAB "port": the second tool's whole purpose is to write a file MATLAB already loads directly via `runpsat`.

Chapter 13

Verification Methodology and Consolidated Defect Log

13.1 Cross-language parity

Every Python module with a natural PSAT-side analogue has a MATLAB port, each tested live against MATLAB R2026a. (MATLAB in this development environment must be invoked with its process sandbox disabled; under the default sandbox, `matlab -batch` fails with a license-service communication error even though the license is valid and network connectivity is fine – a tool-sandbox artifact, not a real licensing failure.) Two Python modules are intentionally *not* ported: reinforcement learning (`rl_env.py`) is fundamentally a Python/Gymnasium-ecosystem capability, and document generation (`lab_report_generator.py`) is already well served by `python-docx`, with a parallel MATLAB implementation judged lower value than the other extensions for the time available. The two MATPOWER-format utilities are Python-only infrastructure by design (§12).

Table 13.1: Python/MATLAB extension parity map.

Python module	MATLAB port(s)	Verification
`psat_io.py`	(reader only; no port needed)	Tested, 3 real files
`gas_network.py`	`psat_gas_network.m`	Live, matches to 2 d.p.
`decentralized_opt.py`	`psat_consensus_dispatch.m`, `psat_decentralized_opt.m`	Live, exact match
`stochastic_opf.py`	`psat_chance_constrained_reserve.m`	Live, exact match
`market_planning.py`	`psat_merit_order.m`, `psat_market_planning.m`	Live, exact match (after dual-sign fix)
`fdia_state_estimation.py`	`psat_fdia_demo.m`	Live, J unchanged under attack
`mpc_export.py`	`psat_discretize.m`, `psat_mpc_design.m`	Live, exact LQR-gain match
`rl_env.py`	*not ported (by design)*	–
`visualization.py`	`psat_visualize.m`	Live
`three_phase_distribution.py`	`psat_three_phase_distribution.m`	Live, 4 d.p. match after transpose fix
`short_circuit.py`	`psat_short_circuit.m`	Live, exact digit match

Python module	MATLAB port(s)	Verification
`battery_storage.py`	`psat_battery_storage.m`	Live, exact match, no bugs
`robust_opf.py`	`psat_robust_opf.m`, `psat_robust_multi_uncertainty.m`	Live, exact match
`cascading_failure.py`	`psat_cascading_failure.m`	Live, exact match
`wide_area_control.py`	`psat_wide_area_control.m`	Live, both instability and fix reproduced
`attack_surface.py`	`psat_attack_surface.m`	Live on WSCC-9
`gnn_power_flow.py`	`psat_gnn_power_flow.m`	Live, trained on 200 real PSAT NR solves
`most_scheduling.py`	`psat_most.m`	Live, cost match to the penny
`stability_constrained_opf.py`	`psat_stability_constrained_opf.m`	Live, exact match incl. 90° threshold
`probabilistic_power_flow.py`	`psat_probabilistic_power_flow.m`	Live, 4 d.p. match
`dynamic_line_rating.py`	`psat_dynamic_line_rating.m`	Live, exact digit match
`lab_report_generator.py`	*not ported (by design)*	–
`matpower_m_parser.py`	*(Python-only infrastructure)*	–
`matpower_to_psat.py`	*(Python-only infrastructure)*	–
(none – new device)	`@ESSclass`	Live, PSAT core integration
(none – new device)	`@GFMclass`	Live, PSAT core integration

13.2 Consolidated defect log

Table 13.2 lists every genuine defect found and fixed during development, in rough chronological/thematic order, kept deliberately blunt about root cause since the recurring *categories* are the real lesson: sign-convention errors (3 independent occurrences across two languages and two solvers), the slack-bus-left-unconstrained DC-OPF formulation error (documented once, then independently re-triggered in two later, unrelated modules), and MATLAB-specific pitfalls (`'` as conjugate transpose, nested functions closing over outer data, local functions not externally callable) that would trip up any future port in this ecosystem.

Table 13.2: Consolidated defect log.

Module	Defect and fix
`psat_results.m`	`DAE.y` orders angles before voltage magnitude, opposite of a first assumption; caught via a known slack-voltage value.
`psat_n1_classify.m`, `psat_secure_atc.m`	Editing `Line.con` in memory and re-calling `runpsat('pf')` has no effect (PSAT re-reads from disk every call); fixed by writing a physically modified file to disk.

Module	Defect and fix
`psat_write_line_outage_file.m`	Three sub-bugs: overwrote the wrong trailing column on a 15-field row; a MATLAB matrix literal required uniform row width; the correct default pad value for an omitted status column is 1 (in service), not 0.
`psat_export_ss.m`	No PSAT routine `fex_abcd` exists; the real routine is `fm_abcd`, with a different (global-struct) calling convention entirely.
`psat_ss_loader.py`	MATLAB's `A(:)` flattens column-major; a first draft reshaped row-major, silently transposing non-square matrices.
`@ESSclass`, `@GFMclass`	A sign-convention error in how generation is injected into PSAT's power-balance equation caused a genuine small-signal instability (eigenvalue +25.9), found via `sssa`, and found to also affect the already-shipped `@ESSclass`.
`decentralized_opt.py`	Fixed step size 0.5 caused λ to diverge; a coarse grid search (not Brent's method) in the ADMM inner problem left a quantization bias; stopping on aggregate balance alone can mask non-consensus among agents.
`market_planning.py` (CVXPY)	Raw CVXPY dual for the balance equality returned the *negative* of the correct LMP.
`psat_market_planning.m` (`linprog`)	The identical dual-sign bug, independently re-discovered in a different language and solver; fixed the same way (negate, then cross-check against the known-correct uncongested price).
`most_scheduling.py`	A quadratic objective is silently rejected by both MILP solvers used; a genuine LODF-formula bug (spurious reactance factor) let a "security-constrained" dispatch violate its own limit by 18 MW.
`most_scheduling.py`'s own test script	An independently-written cvxpy cross-check reproduced the slack-bus-left-unconstrained DC-OPF bug already documented from this course's own Week 6 development – found again, in a brand-new script, months later.
`stability_constrained_opf.py`	`economic_dispatch()` crashed with `TypeError` on genuine infeasibility instead of reporting it; fixed with an explicit `None` check.
`psat_stability_constrained_opf.m`	A correct but cosmetically noisy `quadprog` call on an all-zero Hessian (a pure LP); switched to `linprog`, identical results, no warning.
`psat_gas_network.m`	MATLAB nested functions closing over outer data produced silent `NaN`/singular-Jacobian results despite identical arithmetic working as a plain script; fixed with ordinary local functions and explicit arguments.
`psat_three_phase_distribution.m`	MATLAB's `'` is conjugate transpose for complex arrays, silently swapping phases B and C; produced a fully converged, wrong answer with no error.
`test_psat_dlr.m`	First draft called a MATLAB *local* function from outside its own file (not externally callable); fixed by testing the externally observable behaviour instead.
`gnn_power_flow.py`	An un-normalized shared loss across two outputs of very different magnitude let one output dominate every gradient step, producing a model worse than a trivial baseline on the other output.
`wide_area_control.py`	Applying the consensus weight matrix once per physical step, rather than repeatedly, is unstable (diverges to tens of thousands of Hz), not merely slow.

Chapter 14

Known Limitations

- **MATLAB sandbox dependency.** MATLAB/PSAT must be invoked with the development environment's process sandbox disabled; this is a tooling artifact of this specific environment, not a PSAT or MATLAB licensing limitation.
- **Device classes, v1 scope.** `@ESSclass` and `@GFMclass` both omit reactive-power support and use a fixed (non-time-varying, non-droop) reference signal; natural v2 extensions are noted in each class's own README.
- `dynamic_line_rating.py`'s air-property coefficients are general ideal-gas physics, not independently verified against IEEE 738's own specific published tables; results should be treated as representative, not as a drop-in certified calculation.
- `gnn_power_flow.py` does not generalize across network topologies; a surrogate trained on one topology must be retrained for another.
- `most_scheduling.py` and its MATLAB port are DC-network-only (matching MOST's own current released scope); ramping *costs* (as distinct from ramping *limits*) are not separately priced.
- `matpower_to_psat.py` converts power-flow/OPF data only; it does not fabricate dynamic (inertia, AVR, governor) data, so converted cases are not ready for time-domain or small-signal analysis without further, separately sourced data.
- **No live-Canvas or production-grid validation.** Every numeric result in this report was verified against an independent method, solver, or closed-form result within this development environment; none of these tools have been validated against a live utility SCADA/EMS system or a production grid deployment.

Chapter 15

Conclusion and Future Work

This report has documented 25 Python modules, 31 MATLAB functions, and two genuine new PSAT device classes, closing every gap identified in this project's own *MATPOWER vs. PSAT Capability Comparison Report* and in two subsequent rounds of "what else would be valuable for modern power systems research" review. Every capability was verified against an independent method before acceptance, and every genuine defect found along the way – 19 of them, spanning sign conventions, MATLAB-specific semantics, and optimization-formulation pitfalls – is recorded in Table 13.2 rather than silently corrected and forgotten, because several of these defect *classes* recurred more than once and are worth a future developer's attention before, not after, they recur a third time.

Natural next steps, in rough priority order: (1) time-varying/droop reference signals and reactive-power support for `@ESSclass`/ `@GFMclass`; (2) an AC (rather than DC-only) extension to `most_scheduling.py`, mirroring MATPOWER MOST's own in-development roadmap; (3) a verified reproduction of IEEE 738's own specific air-property tables for `dynamic_line_rating.py`; (4) a cross-topology-generalizing GNN architecture (e.g. explicit per-unit/topology-invariant feature engineering) for `gnn_power_flow.py`; and (5) validation of the Canvas QTI and Mentimeter deliverables documented elsewhere in this project against a live Canvas import, the one verification step this development environment cannot perform on its own.

Bibliography

[1] F. Milano, *Power System Modelling and Scripting*, Springer, 2010. (PSAT 2.1.11 reference manual and source.)

[2] R.D. Zimmerman, C.E. Murillo-Sánchez, R.J. Thomas, "MATPOWER: Steady-State Operations, Planning, and Analysis Tools for Power Systems Research and Education," *IEEE Trans. Power Syst.*, 2011.

[3] R.D. Zimmerman, C.E. Murillo-Sánchez, "MATPOWER Optimal Scheduling Tool (MOST) User's Manual."

[4] D. Bertsimas, M. Sim, "The Price of Robustness," *Operations Research*, 52(1), 2004.

[5] S. Boyd, N. Parikh, E. Chu, B. Peleato, J. Eckstein, "Distributed Optimization and Statistical Learning via the Alternating Direction Method of Multipliers," *Foundations and Trends in Machine Learning*, 2011.

[6] Y. Liu, P. Ning, M.K. Reiter, "False Data Injection Attacks against State Estimation in Electric Power Grids," *ACM CCS*, 2009.

[7] IEEE Standard 738-2012, *IEEE Standard for Calculating the Current-Temperature Relationship of Bare Overhead Conductors.*

[8] J.D. Glover, T.J. Overbye, M.S. Sarma, *Power System Analysis and Design*, Cengage Learning, 6th ed., 2016.

[9] P. Kundur, *Power System Stability and Control*, McGraw-Hill, 1994.

[10] S. Boyd, L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004.

[11] D.P. Bertsekas, *Dynamic Programming and Optimal Control*, Vol. I–II, Athena Scientific, 4th ed., 2017.

[12] D.P. Bertsekas, *Nonlinear Programming*, Athena Scientific, 3rd ed., 2016.

[13] D. Bertsimas, J.N. Tsitsiklis, *Introduction to Linear Optimization*, Athena Scientific, 1997.

[14] R.A. Horn, C.R. Johnson, *Matrix Analysis*, Cambridge University Press, 2nd ed., 2012.

[15] G.H. Golub, C.F. Van Loan, *Matrix Computations*, Johns Hopkins University Press, 4th ed., 2013.

[16] L. Xiao, S. Boyd, "Fast Linear Iterations for Distributed Averaging," *Systems & Control Letters*, 53(1), 2004.

[17] G. Casella, R.L. Berger, *Statistical Inference*, Duxbury/Cengage Learning, 2nd ed., 2002.

[18] A. Abur, A.G. Expósito, *Power System State Estimation: Theory and Implementation*, CRC Press, 2004.

[19] B.D.O. Anderson, J.B. Moore, *Optimal Control: Linear Quadratic Methods*, Prentice Hall, 1990 (Dover reprint, 2007).

[20] J. Nocedal, S.J. Wright, *Numerical Optimization*, Springer, 2nd ed., 2006.

[21] J.R. Birge, F. Louveaux, *Introduction to Stochastic Programming*, Springer, 2nd ed., 2011.

[22] W.H. Kersting, *Distribution System Modeling and Analysis*, CRC Press, 4th ed., 2017.

[23] C.S. Cheng, D. Shirmohammadi, "A Three-Phase Power Flow Method for Real-Time Distribution System Analysis," *IEEE Trans. Power Syst.*, 10(2), 1995.

[24] J.J. Grainger, W.D. Stevenson Jr., *Power System Analysis*, McGraw-Hill, 1994.

[25] A.J. Wood, B.F. Wollenberg, G.B. Sheble, *Power Generation, Operation, and Control*, Wiley, 3rd ed., 2013.

[26] J. Machowski, J.W. Bialek, J.R. Bumby, *Power System Dynamics: Stability and Control*, Wiley, 2nd ed., 2008.